

Final Submission Check List

Project Title: Smart Parking Occupancy Detection System

Each group member's credit for this project:

Name: Essa Alansari	credit: 25%	Member's Signature: Essa Alansari
Name: Maitham Sulais	credit: 25%	Member's Signature: Maitham Sulais
Name: Fahd Laniyan	credit: 25%	Member's Signature: Fahd Laniyan
Name: Nathan Bradshaw	credit: 25%	Member's Signature: Nathan Bradshaw

1. Before the written deliverables deadline specified by the instructor (5/1), we have submitted, via Brightspace:

Done___ The Final Project Report

Done ___ The Demonstration Video

Done ___ The Team Project Notebook

Done ___ The Poster in PDF format

Done ___ All other items, in PDF format, generated in the course or referenced in the written deliverables (all software source code, copies of references, manuals, schematics, PCB design files, etc.)

Submission Date: May 1, 2026

Smart Parking Occupancy Detection System

Name	Email
Essa Alansari	ealansar@purdue.edu
Maitham Sulais	msulais@purdue.edu
Fahd Laniyan	flaniyan@purdue.edu
Nathan Bradshaw	ncbradsh@purdue.edu

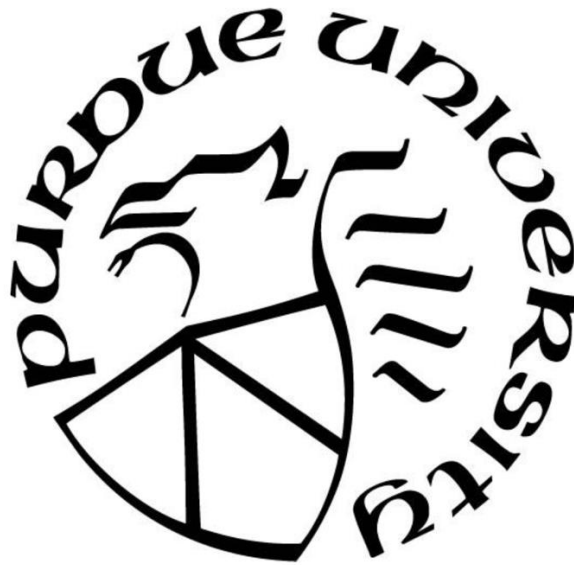


Table of Contents

EXECUTIVE SUMMARY	7
Project Overview	7
Stakeholders	7
FINAL PROJECT REPORT	8
Project Motivation	8
Project Background Information	9
System Block Diagram.....	9
Project Requirements.....	10
Technical Requirements	11
Standards Requirements	12
Safety Requirements	12
Schedule Requirements	12
Other Requirements	13
Constraints	14
System Design Details.....	20
Sensing subsystem	20
<i>Hardware Design</i>	20
<i>Software Design</i>	21
Control Subsystem.....	21
<i>Hardware Design</i>	22
<i>Software Design</i>	22
User Interface Subsystem	23
<i>Hardware Design</i>	23
<i>Software Design</i>	24
Power Management	24
Physical Prototype Construction	25
<i>Hardware Design</i>	26

Software Design	28
Implementation Details.....	28
Testing Procedures, Results, and Analysis	31
Physical Prototype, Power, and Safety Testing.....	32
Final System Test Results	34
Results and Analysis	34
Design Challenges.....	35
ABET Outcomes.....	36
Conclusions and Future Work.....	45
APPENDIX A. SPECIFICATION OF ALL HARDWARE PARTS	48
APPENDIX B. SPECIFICATION OF ALL SOFTWARE AND TOOLS USED	50
APPENDIX C. LIST OF ABBREVIATIONS	51
APPENDIX D. BILL OF MATERIALS.....	52
APPENDIX E. VERIFICATION CROSS REFERENCE MATRIX	54
APPENDIX F. PROOF OF HARDWARE RETURN	62
APPENDIX G. CODE	63

Table of Tables

Table 1 Team roles and current responsibilities.	7
Table 2 Task Timeline	12
Table 3 LCD Decision	14
Table 4 Power supply approach comparison.	15
Table 5 Physical structure material comparison.	16
Table 6 Sensor technology comparison for per-space occupancy detection.	17
Table 7 Wiring topology comparison for sensor and LED interconnect.....	17
Table 8 Current milestone schedule summary.	30
Table 9 Final verification and testing results by requirement.	33
Table 10 Current hardware parts and functions.	48
Table 11 Current software and development tools.	49
Table 12 Bill of materials for the five-space prototype.....	52
Table 13 Verification cross reference matrix.	60
Table 14 Proof of Hardware Return.	61

Table of Figures

Figure 1 Smart parking system architecture for the current five-space scope.....	10
Figure 2 Gantt Chart	13
Figure 3 IR obstacle sensor module used at each parking space (U2–U6).	21
Figure 4 Occupancy detection and indication flow.	22
Figure 5 Conceptual five-space parking layout with per-space indicators and an entrance parking map.....	23
Figure 6 5 mm RGB LED (D1–D5) used as per-space occupancy indicator.	24
Figure 7 Solar panel and power manager assembled with power leads connected.....	27
Figure 8 Solar power manager module with 18650 lithium-ion battery cell.	27
Figure 9 Waveshare Solar Power Manager (U1) — reverse side showing MPPT configuration.	28
Figure 10 Final assembled prototype.....	29
Figure 11 schedule status through wiring and integration.	30
Figure 12 Entrance LCD display	31
Figure 13 solar and battery module	33
Figure 14 Occupancy Detection.....	33

EXECUTIVE SUMMARY

Project Overview

(Essa Alansari – 100%)

The Smart Parking Occupancy Detection System is a compact embedded prototype that detects whether individual parking spaces are occupied or available and communicates the result to drivers through clear visual indicators. The final prototype monitors five parking spaces using IR sensors installed at each space. The system processes the sensor data locally using a Raspberry Pi, then updates the LED indicators and entrance LCD display so drivers can understand parking availability before entering the lot.

The final system combines sensing, central processing, user indication, solar assisted power, and low voltage safety protection in one integrated demonstration platform. Each parking space has its own sensor and LED indicator. The system also includes special status features, including blue indication for handicap parking spaces and orange indication for maintenance or unavailable spaces. These features make the display easier to understand and improve the usefulness of the system.

By the final stage, the team completed the system requirements, VCRM, component selection, physical parking model, sensor mounting, wiring, LCD installation, and system integration. The sensors were mounted in the parking model and tested with model cars. The entrance LCD was mounted and wired using extended connections. The system was tested to confirm that the sensors detect vehicle presence, the visual indicators update correctly, and the display provides parking status information to the user. The project demonstrates a practical smart parking solution that can reduce parking confusion, support better traffic flow, and use solar power for more sustainable operation.

Stakeholders

(Essa Alansari – 100%)

The primary stakeholders are the four senior design team members, the course instructor, and the evaluation audience for the final demonstrations. The team also acts as its own system integrator, which makes clear internal ownership especially important because the project depends on tight coordination between mechanical construction, wiring, sensing, control logic, power, and documentation.

Team member	Current role in project	Primary responsibilities
Essa Alansari	Team lead; system design and demo planning	Coordinates decisions, drives requirements discussions, tracks design

		documentation, and supports physical model planning.
Maitham Sulais	Procurement and documentation support	Supports system planning, purchasing workflow, report development, and design communication.
Fahd Laniyan	Hardware development lead	Owns schematics, GPIO allocation, board wiring, LED/sensor wiring preparation, and electrical integration.
Nathan Bradshaw	Hardware/software integration	Supports controller integration, sensor hookup, firmware bring-up, and transition from wiring to functional testing.

Table 1 Team roles and current responsibilities.

FINAL PROJECT REPORT

Project Motivation

(Essa Alansari – 100%)

Parking inefficiency is a relatively simple problem, yet it has a direct impact on users. Drivers often spend unnecessary time searching for available spaces, traffic congestion increases near parking lot entrances, and the absence of clear availability information leads to avoidable confusion. The team selected a smart parking project because it addresses a practical and recognizable problem while also providing a well-defined systems engineering challenge. The project is appropriately scoped for a senior design course, but it still requires meaningful design tradeoffs across sensing, control, power management, safety, and user interface development.

The project also serves as a strong embedded systems application because it requires the conversion of raw sensor data into a reliable and easily understood parking status indication. For such a system to be effective, it must respond quickly, remain readable to users, operate safely, and reduce false detections. These needs guided the development of the system requirements, the physical model design, and the decision to use both local parking-space indicators and an entrance summary display. In addition, the system was designed to be environmentally friendly by relying on solar power and

rechargeable energy storage, which supports sustainable operation while also introducing important considerations in power regulation and battery management.

Project Background Information

(Essa Alansari – 100%)

Prior work on parking occupancy detection spans a broad range of sensor technologies and system architectures. Paidi et al. (2018) surveyed existing approaches and found that magnetometers, ultrasonic sensors, and machine vision are the most widely deployed technologies in closed parking lots, but noted that none of the reviewed applications provided complete real-time occupancy information suitable for guiding drivers to specific vacant spaces [8]. At the per-space sensor level, Marhoon et al. demonstrated a low-cost embedded prototype using infrared sensors and an Arduino-based controller to monitor slot availability and display status on an LCD, confirming that IR-based detection is accurate and affordable for small-scale systems [9]. Baroffio et al. (2015) presented a visual sensor network approach that centralizes analysis on a server connected to multiple cameras, achieving strong accuracy but requiring persistent network infrastructure and significant computational resources [10]. In contrast, the system proposed in this report targets a self-contained, solar-powered configuration with no cloud dependency. The detection is performed locally on an ARM Cortex-M33 microcontroller (Purdue Proton board), and status is indicated directly through on-space RGB LEDs, eliminating the infrastructure requirements identified as barriers in the prior literature. The notebook history shows that the team began by comparing two project ideas, selected smart parking by team vote, and then moved through block-diagram development, requirements definition, VCRM completion, component planning, procurement, schematic creation, and physical-model construction. The concept of operation has remained stable throughout: each parking space is monitored by a proximity sensor, a central controller aggregates the results, and visual indicators report availability in real time.

During concept development, the team discussed multiple coding-platform families, including STM32, Raspberry Pi, and Arduino-class options, before converging on the Purdue Proton board as the central controller. The Proton board is an ARM Cortex-M33 development platform used in ECE 362 at Purdue and provides sufficient GPIO, ADC channels, and processing headroom for the five-space sensing, control, and indication workload. The notebook also records an important user-interface change: an LCD-style entrance display was considered, but the later design review selected an entrance parking map with LEDs instead because it communicates availability more directly and better

matches the physical model. Earlier notebook revisions reference seven spaces; the current prototype scope is five monitored spaces.

System Block Diagram

(Fahd Laniyan – 100%)

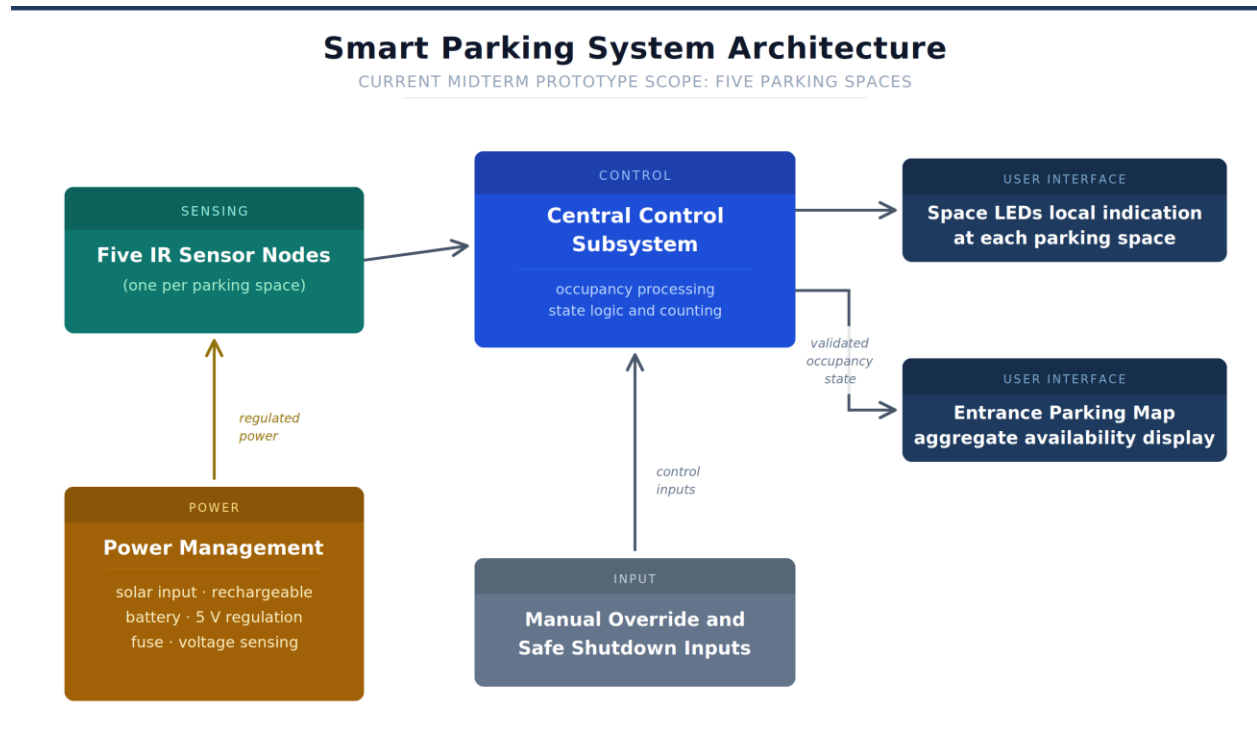


Figure 1 Smart parking system architecture for the current five-space midterm scope.

Figure 1 Smart parking system architecture for the current five-space scope.

Figure 1 shows the system at the subsystem level. Five sensor nodes provide the raw occupancy information. A central control subsystem validates the inputs, maintains the system state, and calculates the total number of available spaces. The user-interface subsystem then presents the result both locally at each parking space and globally at the entrance map. The power-management subsystem supplies regulated low-voltage power through a solar-assisted rechargeable path while also supporting protection features such as fusing and battery-voltage monitoring.

Project Requirements

(Essa Alansari – 100%)

The Smart Parking Occupancy Detection System project aims to develop a reliable and scalable framework for identifying parking space availability in real time, while improving efficiency, usability, and sustainability for parking users. This section outlines the project

requirements by defining the technical, power, safety, and interface features necessary for successful implementation. These requirements were developed through collaboration among team members and were selected to align with the overall project scope, design objectives, and demonstration goals. Together, they establish the foundation for building a system that is accurate, responsive, environmentally friendly, and suitable for practical deployment.

Technical Requirements

The technical requirements define what the system must do. At a minimum, the prototype must detect occupancy at each monitored parking space, drive a corresponding LED indicator, communicate those states to a central controller, calculate the number of available spaces, and update status within ten seconds of a change. The requirements also call for storage of occupancy history, support for a central aggregation function, and local filtering to reduce false detections caused by transient objects or noisy measurements.

Req. 1: The Edge Sensor Node Subsystem shall detect whether its assigned parking space is occupied or empty.

Req. 2: The Edge Sensor Node Subsystem shall control an LED indicator to represent the occupancy status of its assigned parking space.

Req. 3: The Communication Subsystem shall transmit parking availability data from each Edge Sensor Node to the Central Processing Subsystem.

Req. 4: The Central Processing Subsystem shall store historical parking occupancy data for analysis purposes.

Req. 5: The Central Processing Subsystem shall calculate the total number of available parking spaces.

Req. 6: The Edge Sensor Node Subsystem shall interface with a proximity sensor installed at each individual parking space.

Req. 7: The Central Processing Subsystem shall use the Purdue Proton board (ARM Cortex-M33) as the main processing unit.

Req. 8: The Central Processing Subsystem shall process and aggregate occupancy data received from all Edge Sensor Nodes.

Req. 9: The Edge Sensor Node Subsystem shall update occupancy status within 10 seconds of detecting a change.

Req. 10: The Power Management Subsystem shall operate using energy supplied by a solar charging system.

Req. 19: The Power Management Subsystem shall store energy using rechargeable lithium-ion or equivalent rechargeable battery technology.

Req. 20: The Edge Sensor Node Subsystem shall perform local occupancy validation filtering to reduce false detections caused by environmental noise or transient objects.

Standards Requirements

The project includes a documented safety-related standards requirement tied to UL 62368-1. That requirement drives the need for insulation, low-voltage operation, current limiting, protected wiring, and deliberate attention to battery and charging behavior. The VCRM work performed in February explicitly mapped the requirements to subsystems and verification methods, which helps ensure the standard-related requirement is treated as a design driver rather than an afterthought.

Req. 15: The Electrical Design Subsystem shall comply with UL 62368-1 by incorporating proper insulation, current limiting, and protection against electric shock and fire hazards for low-voltage electronic equipment.

Safety Requirements

The safety requirements focus on keeping the prototype electrically benign for classroom demonstration. The power-management subsystem must prevent battery overcharge, keep accessible conductive parts below hazardous voltage levels, support safe shutdown, and allow normal operation while charging. These requirements also influence the structural layout because wiring channels, insulation, fuse placement, and physical compartment separation all reduce the chance of accidental shorts or contact with energized conductors.

Req. 11: The Power Management Subsystem shall ensure that no accessible conductive parts exceed 30V DC under normal operating conditions.

Req. 12: The Power Management Subsystem shall prevent battery overcharge by continuously monitoring battery voltage using a voltage sensing circuit.

Req. 16: The Power Management Subsystem shall support simultaneous battery charging and normal system operation.

Schedule Requirements

The schedule requirements for a Final submission are practical rather than theoretical: the report must document substantive design work, show the logical order of system assembly, and explain how the team is sequencing the remaining tasks so the project can progress from wiring to integration and then to verification. The project notebook demonstrates that the team has already used weekly meetings to turn broad goals into near-term tasks such as schematic completion, wiring preparation, and controller integration.

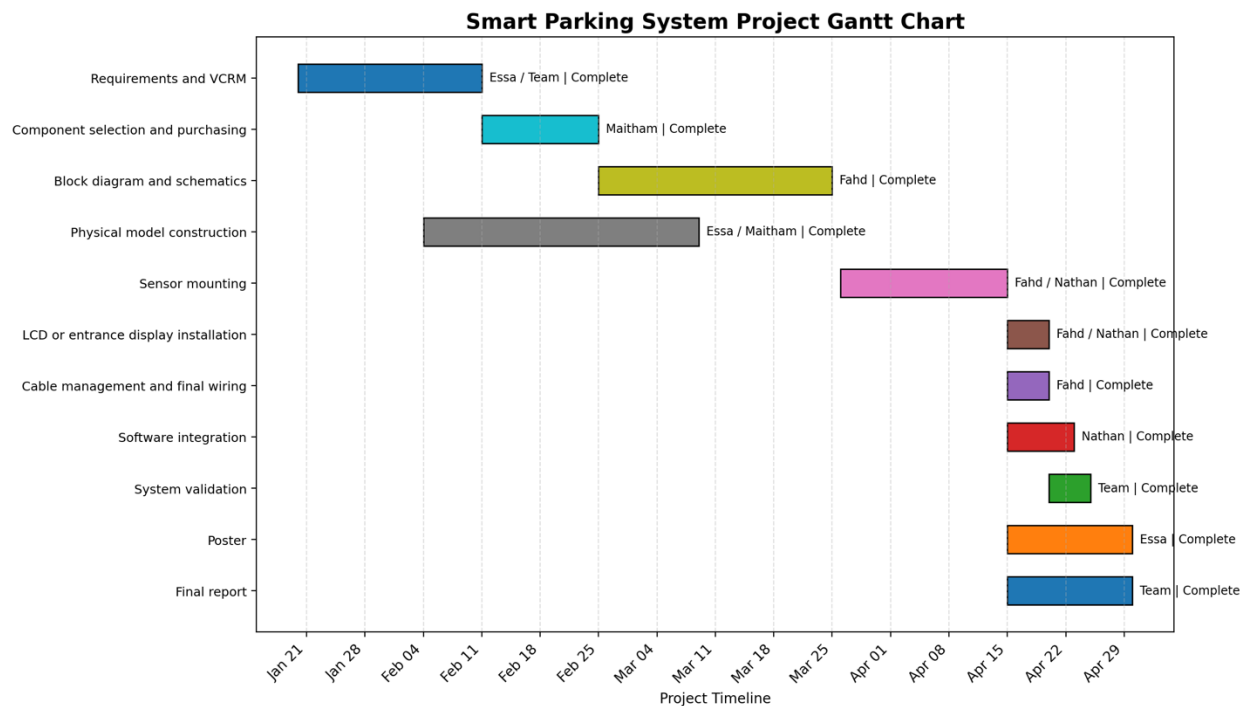


Figure 2 Gantt Chart

Task	Start	End	Owner	Status
Requirements and VCRM	Jan. 20	Feb. 11	Essa / Team	Complete
Component selection and purchasing	Feb. 11	Feb. 25	Maitham	Complete
Block diagram and schematics	Feb. 25	Mar. 25	Fahd	Complete
Physical model construction	Feb. 4	Mar. 9	Essa / Maitham	Complete
Sensor mounting	Mar. 26	Apr. 15	Fahd / Nathan	Complete
LCD or entrance display installation	Apr. 15	Apr. 20	Fahd / Nathan	Complete
Cable management and final wiring	Apr. 15	Apr. 20	Fahd	Complete
Software integration	Apr. 15	Apr. 23	Nathan	Complete
System validation	Apr. 20	Apr. 25	Team	Complete
Final report and poster	Apr. 15	Apr. 30	Essa / Team	Complete

Table 2 Task Timeline

Other Requirements

The remaining requirements improve usability and maintainability. The system must identify handicap spaces separately, support a maintenance indication mode, include

manual override capability for authorized use, and provide a physical shutdown control. Together, these requirements move the prototype beyond simple sensor blinking and toward a more complete parking-management demonstration.

Req. 13: The Central Processing Subsystem shall identify designated handicap parking spaces separately from regular spaces in the display interface.

Req. 14: The Edge Sensor Node Subsystem shall support a maintenance status mode that displays yellow or orange LED indication during construction or servicing.

Req. 17: The User Interface Subsystem shall include a physical shutdown button that safely powers off the Central Processing Subsystem.

Req. 18: The Edge Sensor Node Subsystem shall include a manual override input to allow authorized personnel to change the parking status indication when required.

The complete requirement-to-verification mapping for all twenty requirements appears in Appendix E, which includes the full requirement text, level, type, verification method, rationale, source, and subsystem allocation for each requirement.

Constraints

(Maitham Sulais – 100%)

Three main constraints affected the final design: time, budget, and physical scope. The project had to be completed within one semester, so the team selected a five-space prototype that was large enough to demonstrate the main smart parking functions but still realistic to build, wire, and test before the final deadline. The course budget limit also affected the design because the team needed to choose low-cost sensors, LEDs, power parts, and structure materials while still keeping the prototype functional and presentable.

The physical size of the prototype was another important constraint. The model needed enough space for parking spots, sensors, LEDs, wiring paths, the controller, the entrance display area, and power components. At the same time, it had to remain small enough for classroom demonstration and transportation. This constraint influenced the decision to use corrugated plastic board because it was lightweight, easy to cut, and allowed the team to build side compartments for cable management.

These constraints helped guide the final design choices. The five-space layout reduced wiring complexity while still allowing the team to demonstrate occupancy detection, local indication, entrance guidance, handicap indication, maintenance mode, manual override, and power supervision. The LED-based indication and simple physical layout also kept the system understandable, low cost, and easier to test during final integration.

System Design Options

(Essa Alansari – 25%, Maitham Sulais – 25%, Fahd Laniyan – 25%, Nathan Bradshaw – 25%)

Essa Alansari Design Option Analysis: Entrance User Interface and Display Layout

Option	Pros	Cons	Decision
LCD at entrance	Shows number of available spaces clearly.	Needs more wiring and programming. It may be harder to see from a far distance.	Select only if this is the final design.
LED parking map	Easy to understand visually and matches the physical parking model.	More wires and more LEDs are needed.	Select only if this is the final design.
LCD plus LEDs	Gives the best user information because it combines count and visual location.	Most complex option because it requires both display programming and LED wiring.	Use only if both were actually built.

Table 3 LCD Decision

The entrance user interface was one of the main design decisions for the project because it is the first thing the driver will see before entering the parking lot. The team compared using only an LCD display, using an LED parking map, and combining the LCD with LEDs. Each option had benefits, but the final choice needed to be clear, easy to understand, and practical for the physical prototype.

The LCD at the entrance was useful because it could show the number of available parking spaces clearly. This makes it easy for the driver to know if parking is available before entering. However, using only an LCD does not show the exact location of each available space. It also requires programming and wiring to make sure the display updates correctly based on the sensor readings.

The LED parking map was also a strong option because it gives a visual layout of the parking lot. This helps the driver quickly understand which spaces are open, occupied, handicap, or under maintenance. The main disadvantage is that it requires more wiring because each parking space needs its own LED connection. Even with this disadvantage, the LED map matches the physical parking model well and makes the system easier to understand during the demonstration.

The final design used the LCD display along with LED indicators. This was selected because it gives the user both types of information. The LCD can show the total parking availability, while the LEDs show the status of individual spaces. This design is more complex than using only one interface, but it provides the clearest information for the user and makes the final prototype more complete.

Maitham Sulais Design Option Analysis: Power and Physical Material Selection

My design option analysis focused on the power approach and the physical material used for the prototype structure. For the power system, the main options were wall power only, battery only, and solar power with a rechargeable battery. Wall power would have been the easiest option for testing because it provides a stable supply and does not require charging hardware. However, it would make the prototype less portable and would not support the project goal of using a more sustainable energy source. A battery-only design would make the system portable, but it would still need to be recharged manually and would not show the environmental goal of the project.

Option	Pros	Cons	Selected
Wall power only	Stable supply; simplest implementation	Not portable; no sustainability goal	
Battery only	Portable; no charging hardware needed	Manual recharge required; no renewable energy demonstration	
Solar + battery	Portable; sustainable; operates without wall outlet	Additional wiring; voltage regulation and charging complexity	✓

Table 4 Power supply approach comparison.

The selected option was a solar-assisted rechargeable battery system. This option supports portability, allows the prototype to operate without depending only on a wall outlet, and matches the project motivation of making the system more environmentally friendly. The disadvantage is that it adds more wiring and safety considerations, especially related to voltage regulation, battery charging, and protection. Because of this, the power design also included low-voltage operation, fuse protection, and covered wiring to support safe classroom demonstration.

For the physical structure, several material options were considered, including wood, acrylic, cardboard, and corrugated plastic board. Wood and acrylic would provide stronger structures, but they are harder to cut, heavier, and less convenient for fast prototype changes. Cardboard would be easy to cut, but it would not be strong enough for a clean final prototype. Corrugated plastic board was selected because it is lightweight, low cost, easy to cut, and strong enough for the parking model. It also allowed the team to build the parking base, entrance display frame, and side compartments for wire management in a simple and organized way.

Option	Pros	Cons	Selected
Wood	Rigid; strong	Heavy; difficult to cut; less flexible for design changes	
Acrylic	Rigid; clean appearance	Brittle; harder to cut; higher cost	
Cardboard	Very easy to cut; low cost	Not durable enough for classroom prototype use	
Corrugated plastic	Lightweight; low cost; easy to cut; strong enough	Less rigid than wood or acrylic	✓

Table 5 Physical structure material comparison.

The final decision was to use a solar-assisted rechargeable battery system for power and corrugated plastic board for the physical prototype. These choices supported the project goals of portability, sustainability, safety, and clean physical organization.

Fahd Laniyan Design Option Analysis: Sensor and Wiring Design

My hardware analysis covered two decisions that shaped most of the physical build: what sensor to use at each parking space, and how to run the wiring back to the controller. These two choices were connected because the sensor type influenced how many conductors were needed, and the wiring approach depended on how each sensor interfaced with the Proton board.

For the sensor, three options were evaluated: infrared obstacle modules, ultrasonic distance sensors, and passive infrared motion sensors. Ultrasonic sensors can measure distance and would allow a detection threshold to be set at a fixed height above the ground, but they are more susceptible to environmental noise and require either analog processing or a timed pulse measurement on the controller side. PIR sensors were straightforward to eliminate because they detect movement, not static presence, so a parked car sitting still would not register reliably. IR obstacle modules ended up being the right fit. Each one outputs a clean high or low digital signal that the Proton reads directly off a GPIO pin with no additional conditioning needed. The onboard sensitivity potentiometer also made bench testing easier, since the detection range could be dialed in for the size of the model cars without touching firmware. The tradeoff is a limited detection cone, which means mount position matters. This was addressed during installation by standardizing the height and angle across all five spaces.

Option	Pros	Cons	Selected
IR obstacle module	Clean digital output; on-board sensitivity pot; 3.3 V compatible; low cost	Limited range; mount angle sensitive	✓
Ultrasonic sensor	Distance measurement; adjustable threshold	Noise susceptibility; requires timed processing	
PIR motion sensor	Low power; wide field of view	Detects motion only; unreliable for stationary vehicles	

Table 6 Sensor technology comparison for per-space occupancy detection.

For wiring, three approaches were considered: point to point runs from each sensor and LED back to the board, a shared addressing bus, and a breakout board. A bus would reduce wire count but requires address hardware on each node and matching firmware to decode it, which is more complexity than the five space scope needed. A breakout board would have cleaned up the connection points but added another component to source and integrate into the layout. Point to point was the right call given the prototype scale: every channel is independent, easy to trace, and easy to verify one space at a time. The higher wire count was handled by routing all conductors through the lower and side compartments of the model and grouping them by space number before terminating at the board.

Option	Pros	Cons	Selected
Point to point	Independent channels; easy per-space debug; no added hardware	Higher wire count; requires organized routing	✓
Shared address bus	Lower wire count	Requires addressing hardware and firmware overhead	
Breakout board	Cleaner connector interface	Additional cost and board layout complexity	

Table 7 Wiring topology comparison for sensor and LED interconnect.

The final choices were IR obstacle modules for detection and organized point to point wiring, which kept the hardware straightforward and made integration and channel level troubleshooting easier through the final build stages.

The software for the smart parking system was developed in C using the Raspberry Pi Pico SDK. The firmware is structured around a single main polling loop that executes every 50 milliseconds. Within each loop iteration, the firmware reads button inputs, queries the IR occupancy sensors, updates the RGB LEDs, conditionally refreshes the LCD display, and periodically outputs diagnostic statistics to the serial console. Each of the five parking spaces is independently tracked using three state variables: its space type (Normal, Handicap, or Unavailable), whether an override is currently active, and what the override occupancy value is set to. Occupancy is determined by reading the IR sensor GPIO pins (which are configured as active-low inputs, meaning a logic-low signal indicates a vehicle is present). This raw sensor state can be overridden by an operator via the physical button assigned to each space, allowing staff to manually correct false readings or flag a space occupied for non-vehicle reasons without altering the underlying sensor hardware.

The button handling logic implements two distinct behaviors differentiated by press duration. A short press toggles a manual override for that space, flipping the displayed occupancy state independently of the sensor. If the space is marked Unavailable, a short press restores it to its previously saved type instead. A long press held for at least 1,500 milliseconds (about 3 seconds) toggles the space between its current type and the Unavailable state, which is used to indicate spaces under maintenance or otherwise out of service. When a space is set to Unavailable, its prior type is saved so it can be restored correctly when the space returns to service. The RGB LED indicators provide immediate local feedback at each physical space. The color mapping follows an intuitive convention: green indicates a standard space is open, blue indicates a handicap space is open, red indicates the space is occupied, and yellow indicates the space is unavailable or under maintenance. LED state is determined after applying any active override, so the physical indicator always reflects what the system considers the effective status of that space.

The ILI9341 TFT LCD is driven over SPI at 62.5 MHz. Each space occupies a dedicated row showing its label alongside a color-coded status badge reading OPEN, FULL, or N/A. A summary line at the bottom of the screen shows the real-time count of available spaces out of total usable spaces. To avoid unnecessary redraw latency, the firmware uses a selective update function that only redraws screen regions for spaces whose displayed state has changed since the last frame, rather than performing a full screen refresh on every loop iteration. A custom 8×8 pixel bitmap font is rendered directly via SPI, avoiding any external display library dependency. The firmware also incorporates a lightweight analytics subsystem that tracks per-space occupancy statistics over time. Whenever a

space transitions from free to occupied, the system records the start time using the Pico's hardware timer. When the space becomes free again, the elapsed duration is added to a cumulative occupied-time accumulator and the session count is incremented, provided the duration exceeds one second to filter out noise. Every five seconds, these statistics are printed to the serial console in a formatted table showing each space's current status, average occupancy duration in seconds, and total session count. Additionally, the system identifies and recommends a free standard space to incoming drivers, preferring the space with the highest average occupancy duration and falls back to the first available normal space if no historical data exists.

System Design Details

(Fahd Laniyan – 50%, Nathan Bradshaw – 50%)

Sensing subsystem

(Fahd Laniyan – 50%, Nathan Bradshaw – 50%)

The sensing subsystem is responsible for determining whether each monitored parking space is occupied. In the current prototype, the subsystem is organized as one sensor per parking space, so the design scales linearly with lot size and remains easy to reason about during troubleshooting. Each sensing channel interfaces upward to the central controller, sideways to the local LED indication, and physically downward into the under-board cable routing space built into the parking model.

Hardware Design

The hardware design places one IR obstacle sensor module at each of the five parking spaces (U2 through U6 in the KiCad schematic). Each module outputs a digital signal (IR_SLOT1_OUT through IR_SLOT5_OUT) that the Proton board reads on a dedicated GPIO pin. The module operates on the +3V3 rail and provides a clean active-low or active-high digital output, avoiding the need for external analog conditioning. This one-sensor-per-space topology simplifies the channel-to-space mapping and makes troubleshooting straightforward because each input can be exercised independently. The wiring routes through the under-board cable channel of the parking model, protecting conductors and maintaining a clean visible surface.



Figure 3 IR obstacle sensor module used at each parking space (U2–U6).

Software Design

From a control perspective, the sensing subsystem should not pass every raw input change directly to the user interface. The requirements explicitly include local occupancy validation filtering, which means the software should sample, debounce, and validate sensor state before updating the system status. This prevents brief transients from generating visible flicker or incorrect lot counts.

Control Subsystem

(Fahd Laniyan – 50%, Nathan Bradshaw – 50%)

The control subsystem serves as the decision center of the prototype. It receives each space-level input, validates the states, calculates the available-space count, distinguishes handicap status from regular status, handles maintenance and manual-override conditions, and issues the output commands that drive the visual indication. It is also the natural place to store occupancy history because it already owns the system-wide state table.

Hardware Design

The Purdue Proton board (ARM Cortex-M33) is the selected central controller. It exposes 48 GPIO pins and multiple ADC channels, which is more than sufficient to read all five IR sensor outputs (IR_SLOT1_OUT through IR_SLOT5_OUT), drive the fifteen current-limiting resistors feeding the five RGB LED indicators (D1–D5, three channels each), accept five manual-override pushbutton inputs (BTN1–BTN5), and respond to the combined shutdown/mode reset button (BTN_MODE_RESET). The voltage-sensing circuit (J2) connects to an ADC channel (VBAT_SENSE) for battery monitoring. The completed KiCad schematic documents the full GPIO allocation across the Proton header left (J4) and header right (J5) connectors, and physical board wiring began in late March following schematic completion.

Software Design

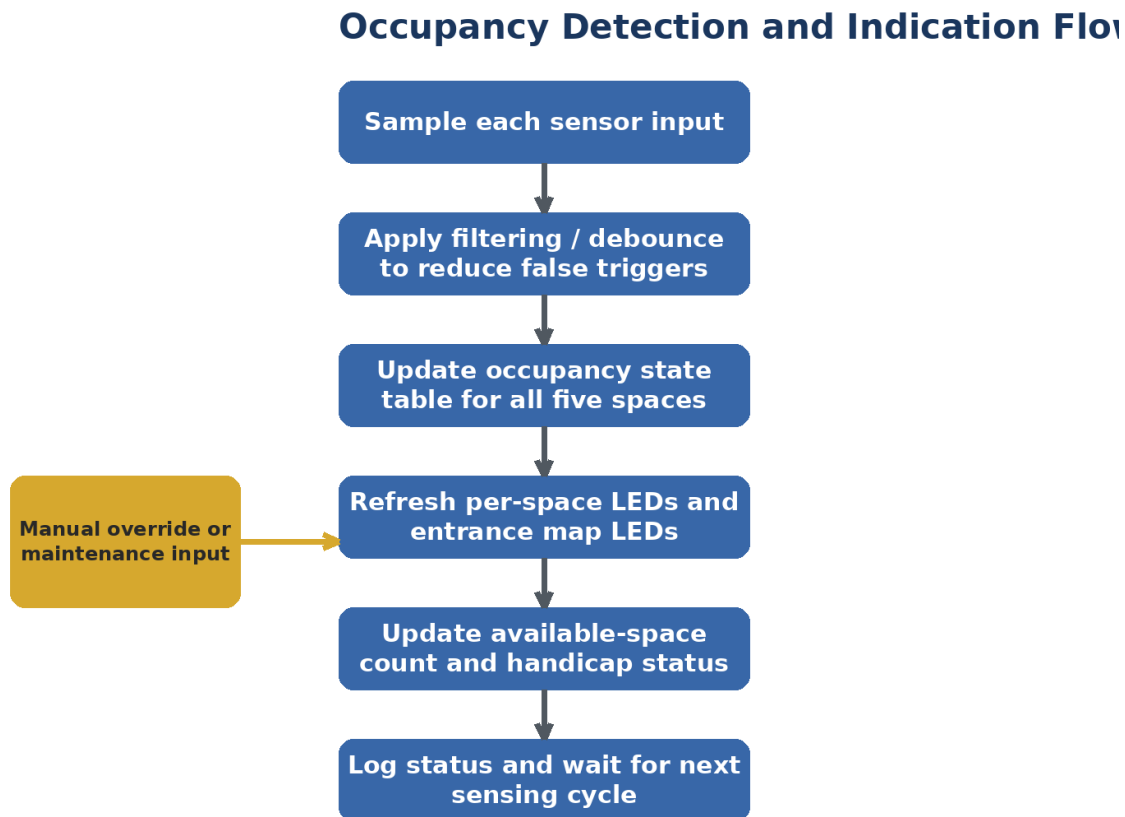


Figure 4 Occupancy detection and indication flow.

The control software can be described as a repetitive scan-and-update loop. Each cycle samples the sensor inputs, filters the readings, updates the occupancy state table, refreshes the local LEDs and entrance map, recomputes the number of available spaces,

and records the resulting status. Figure 3 captures that flow at the logic level. The software must also check for special conditions such as maintenance mode, manual override, and safe shutdown requests.

User Interface Subsystem

(Fahd Laniyan – 50%, Nathan Bradshaw – 50%)

The user-interface subsystem is intentionally visual and low cognitive load. One layer of indication sits directly at each parking space, while a second layer sits at the entrance in the form of a parking map. This dual representation reduces ambiguity: a driver can first understand lot-level availability before entering and then still read the status of each individual space while approaching it.

Hardware Design



Figure 5 Conceptual five-space parking layout with per-space indicators and an entrance parking map.

The March 9 meeting finalized the user-interface design, with the team selecting a parking map with LED indicators as the preferred option over an LCD-style entrance display. The schematic implements five RGB LED indicators (D1–D5), one per parking space, with the R prong driven through a dedicated 330 Ω current-limiting resistor, and the G and B prongs driven through 33 Ω current-limiting resistors. These resistor values limit each channel to a safe current level relative to the +3V3 supply and the LED forward voltage. The RGB

capability allows the firmware to show green for available, red for occupied, yellow or orange for maintenance mode, and a distinct color or pattern for handicap spaces — all from a single physical LED per space. Entrance map indication is mirrored from the same GPIO signals so the driver at the lot entrance sees the same status as the space-level indicators without requiring separate hardware.



Figure 6 5 mm RGB LED (D1–D5) used as per-space occupancy indicator.

Software Design

The UI update logic should separate state generation from rendering. The controller first computes the current truth state of each space, then the UI layer maps that state into color or on/off patterns for both the local LEDs and the entrance map. That separation matters because it keeps the visual layer simple and makes it easier to add future interface changes without rewriting the sensing and decision logic.

Power Management

(Maitham Sulais – 100%)

The power subsystem was included to make the prototype safer, more portable, and more connected to the sustainability goal of the project. Instead of depending only on a wall outlet, the final design used a solar-assisted rechargeable battery approach. This allowed the system to demonstrate how a smart parking prototype could operate using stored renewable energy while still providing regulated low-voltage power to the controller, sensors, and LED indicators.

The power design included solar charging hardware, a rechargeable battery, voltage regulation, fuse protection, and covered wiring. These parts were important because the system needed to operate safely during classroom demonstration. The design kept the accessible electrical parts at low voltage and used protected wiring paths to reduce the risk of accidental shorts or exposed connections. This supported the safety intent of UL 62368-1 by using insulation, current limiting, and low-voltage operation.

Physical Prototype Construction

(Maitham Sulais – 80%, Essa Alansari 20%)

The physical prototype was built to support the parking layout, sensor placement, entrance display area, and internal cable routing. The main structure was made from corrugated plastic board because it was lightweight, low cost, and easy to cut during fabrication. This material was strong enough for the final classroom prototype while still allowing changes during assembly.

The final model included a flat parking base, an entrance display frame, and lower side compartments for wiring and sensor organization. The parking base was approximately 46 cm long, the entrance frame was approximately 27 cm high, and the LED/sensor compartment was approximately 9 cm deep and 7.5 cm high. The side compartments were raised compared with the original concept so the team had more space for wires, sensors, and internal hardware. This made the prototype easier to assemble and helped keep the visible parking area cleaner.

The physical structure also helped with safety and presentation. Wires were routed through the lower and side compartments instead of being left exposed on the top surface. This made the prototype look more organized and reduced the chance of wires being pulled or damaged during testing. The final structure provided a clear separation between the visible parking area and the internal hardware, which improved both appearance and reliability during the final demonstration.

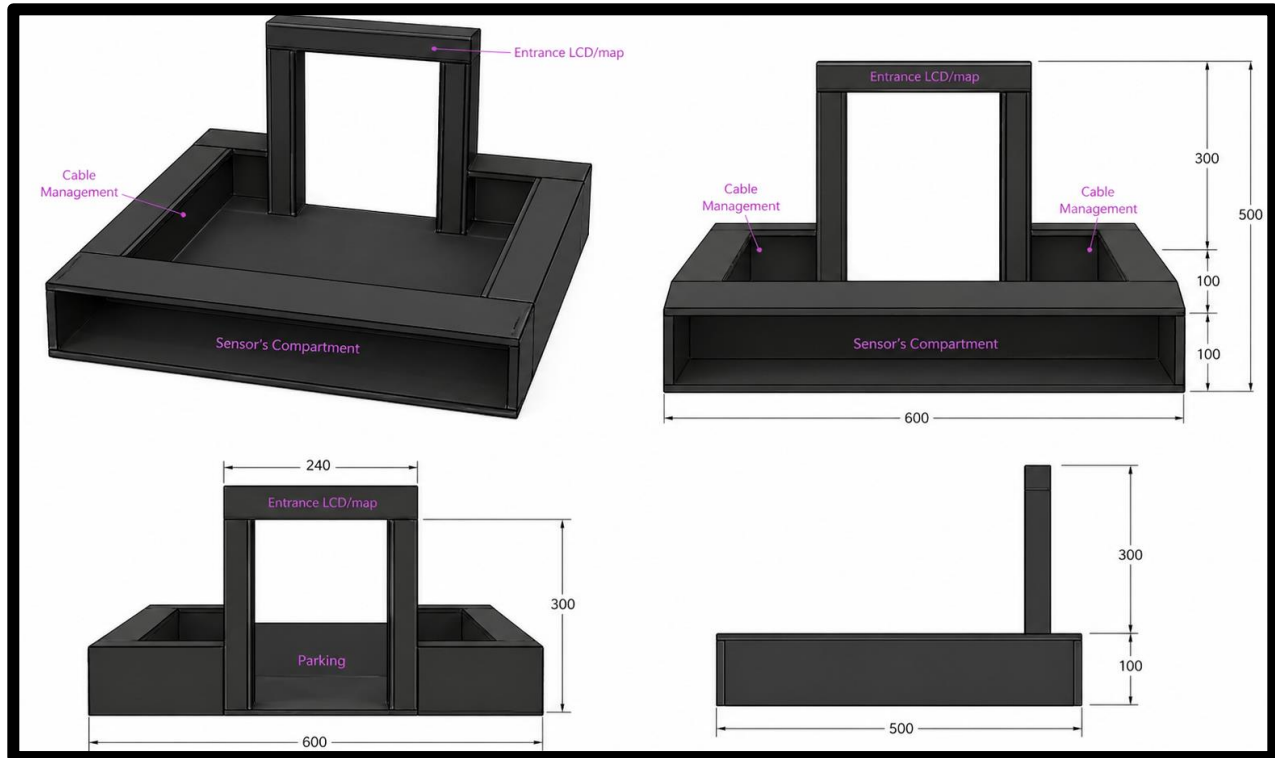


Figure 7 Smart Parking System CAD Design

Hardware Design

The power hardware in the schematic consists of a solar panel connector (J3, label SOLAR_IN), a solar power manager module (U1, which handles MPPT charging and regulated output), a rechargeable lithium-ion battery pack (BATT_POS rail), a 5 V regulator providing the VBUS rail, and a voltage-sensing circuit (J2, connected to the VBAT_SENSE ADC label on the Proton) for continuous battery monitoring. The fuse is placed in the power path to provide overcurrent protection consistent with the UL 62368-1 compliance intent. All logic and peripheral power runs on the +3V3 rail derived downstream of VBUS, keeping every accessible conductor well below hazardous voltage levels. The combination of solar input, battery storage, and regulated 3.3 V logic supply allows the system to operate continuously during a demonstration without dependence on a wall outlet, and supports simultaneous charging and operation as required.

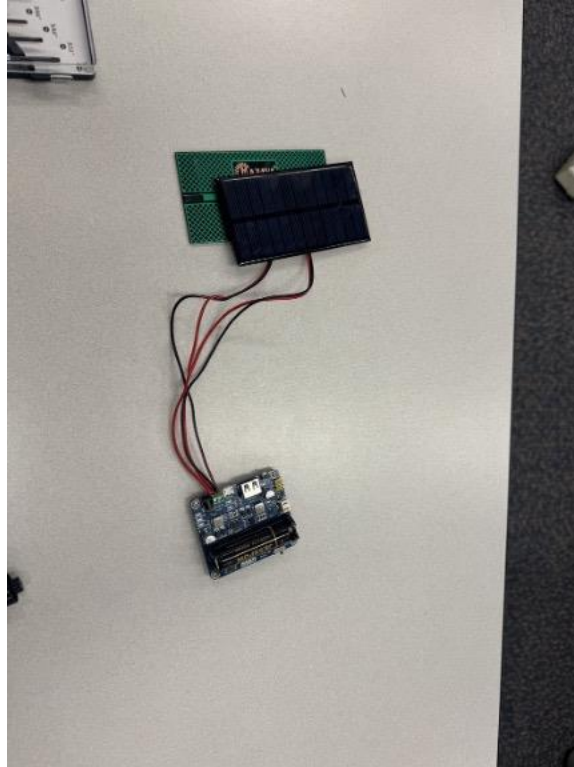


Figure 7 Solar panel and power manager assembled with power leads connected.

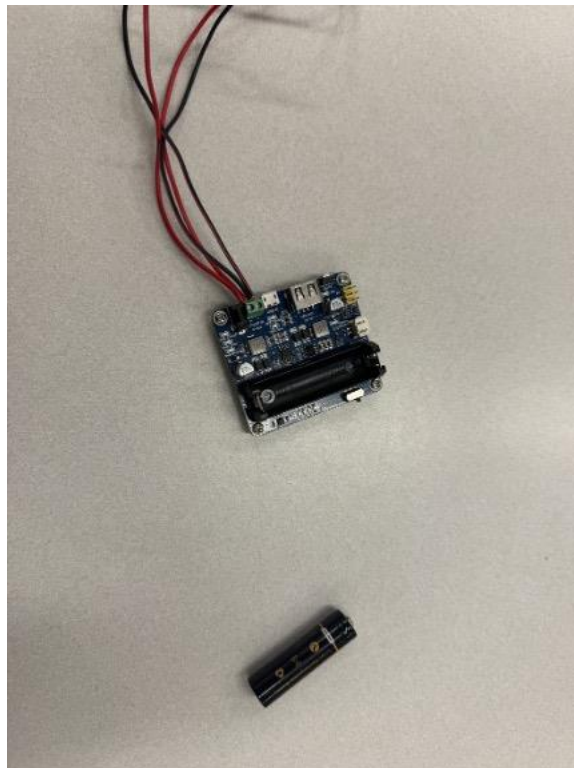


Figure 8 Solar power manager module with 18650 lithium-ion battery cell.

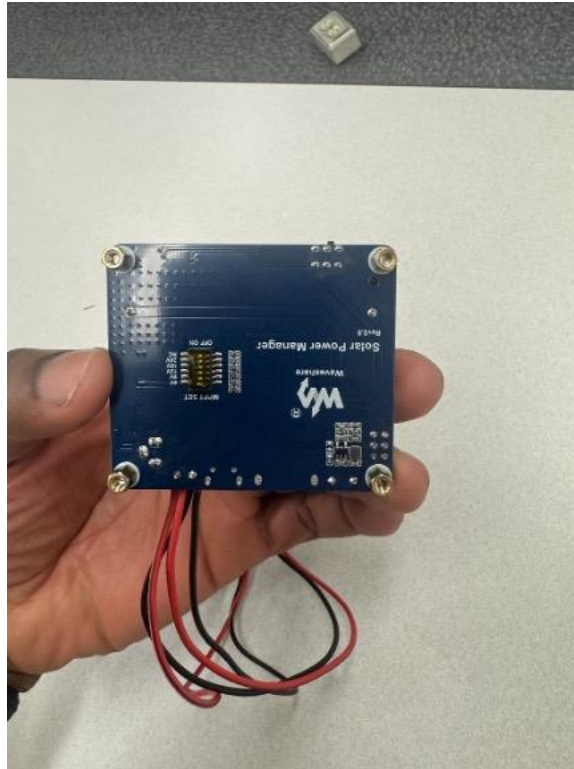


Figure 9 Waveshare Solar Power Manager (U1) — reverse side showing MPPT configuration.

Software Design

The power subsystem requires only lightweight supervisory logic compared with the sensing and control paths. The essential software behavior is to monitor battery voltage, support safe shutdown behavior, and avoid control actions that violate the protected operating envelope of the system. As hardware integration progresses, these supervisory checks should be validated together with the main occupancy logic rather than treated as a separate late-stage task.

Implementation Details

(Essa Alansari – 25%, Maitham Sulais – 25%, Fahd Laniyan – 25%, Nathan Bradshaw – 25%)

Physical Prototype and Power Integration

(Maitham Sulais – 50%, Essa Alansari – 50%)

The physical prototype was assembled first so the rest of the system would have a clear mechanical structure for mounting and wiring. The parking base was built from corrugated

plastic board and divided into five parking spaces. The entrance display frame was attached at the front of the model so drivers could view the parking status before entering the lot. Side and lower compartments were added to keep the wiring, sensors, and power connections separated from the visible parking surface.

After the base structure was completed, the sensors and LED indicators were arranged according to the five-space layout. Each parking space had a dedicated sensor area and a matching visual indicator. The wiring was routed through the side compartments so that the top surface stayed clean and easier to understand during demonstration. This also helped protect the wires from being pulled or damaged when the model cars were moved during testing.

The power system was integrated after the physical structure and wiring paths were prepared. The solar charging hardware, rechargeable battery path, voltage regulation, and fuse protection were placed in the internal hardware area. The system was designed to provide low-voltage power to the controller, sensors, and display components. The wiring was kept covered as much as possible to support safe classroom handling and to reduce the risk of accidental short circuits.

This implementation order helped the team avoid major layout changes later in the project. By building the physical structure first and planning the cable paths early, the electrical and software integration could be completed with a more organized layout. It also made troubleshooting easier because each parking space had a clear sensor, LED, and wiring path connected to the controller.

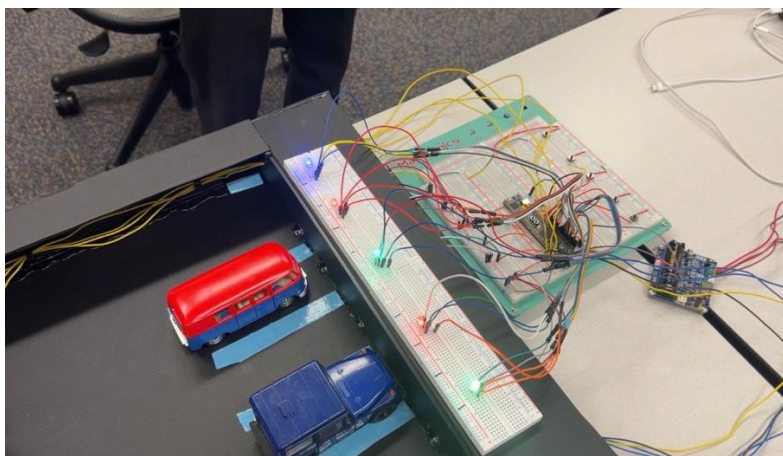


Figure 10 Final assembled prototype

This figure shows the final prototype after the parking model, sensors, LED indicators, controller wiring, and model vehicles were integrated. The image demonstrates how the

physical parking layout connects to the electronic control hardware for the final system build.

In addition, it shows the controller, breadboard wiring, LED indicators, and sensor connections used during system integration. The wiring connects the parking model sensors and display outputs to the central controller so the system can process occupancy data and update the indicators.

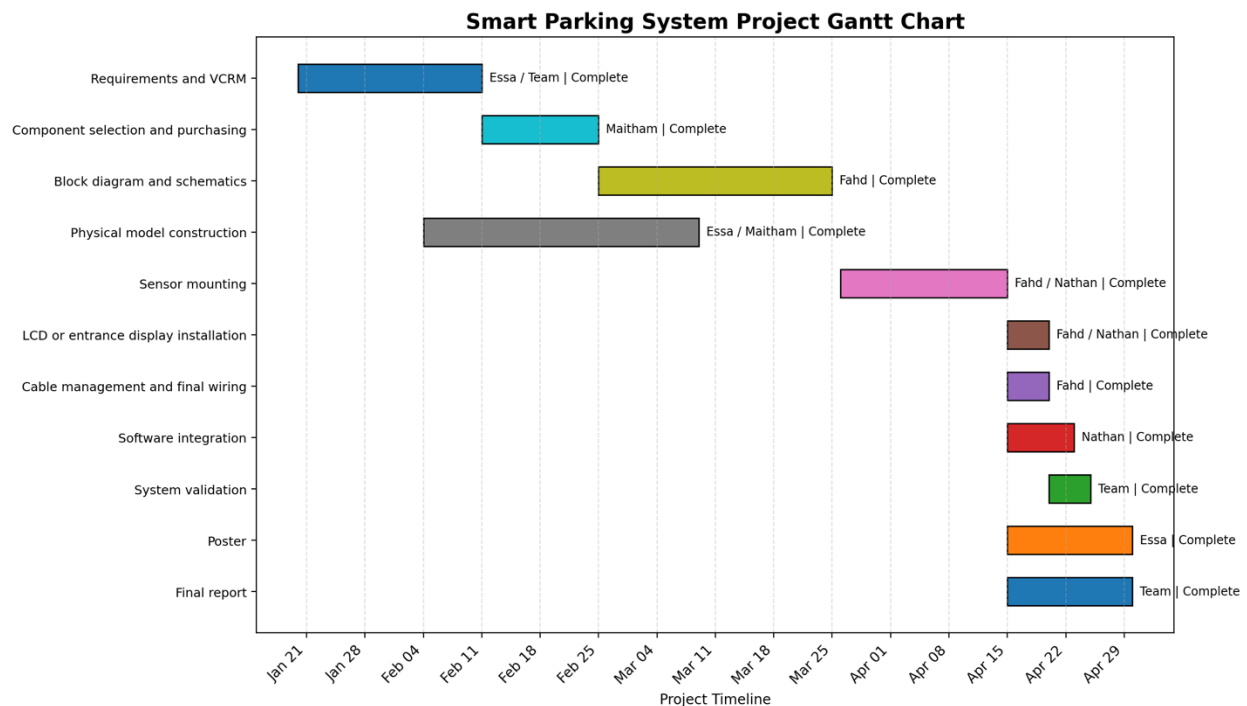


Figure 11 schedule status through wiring and integration.

Milestone	Date window	Status
Requirements and CONOPS complete	Jan. 20 – Jan. 28	Done
VCRM and safety mapping complete	Feb. 4 – Feb. 11	Done
Component selection and procurement setup	Feb. 11 – Feb. 25	Done
Schematics and GPIO allocation	Feb. 25 – Mar. 25	Done
Physical model construction	Feb. 4 – Mar. 9	Done
Board wiring	Late March – April 25	Done

Sensor integration and software bring-up	April 10	Done
System verification and demo preparation	April 20	Done

Table 8 Current milestone schedule summary.

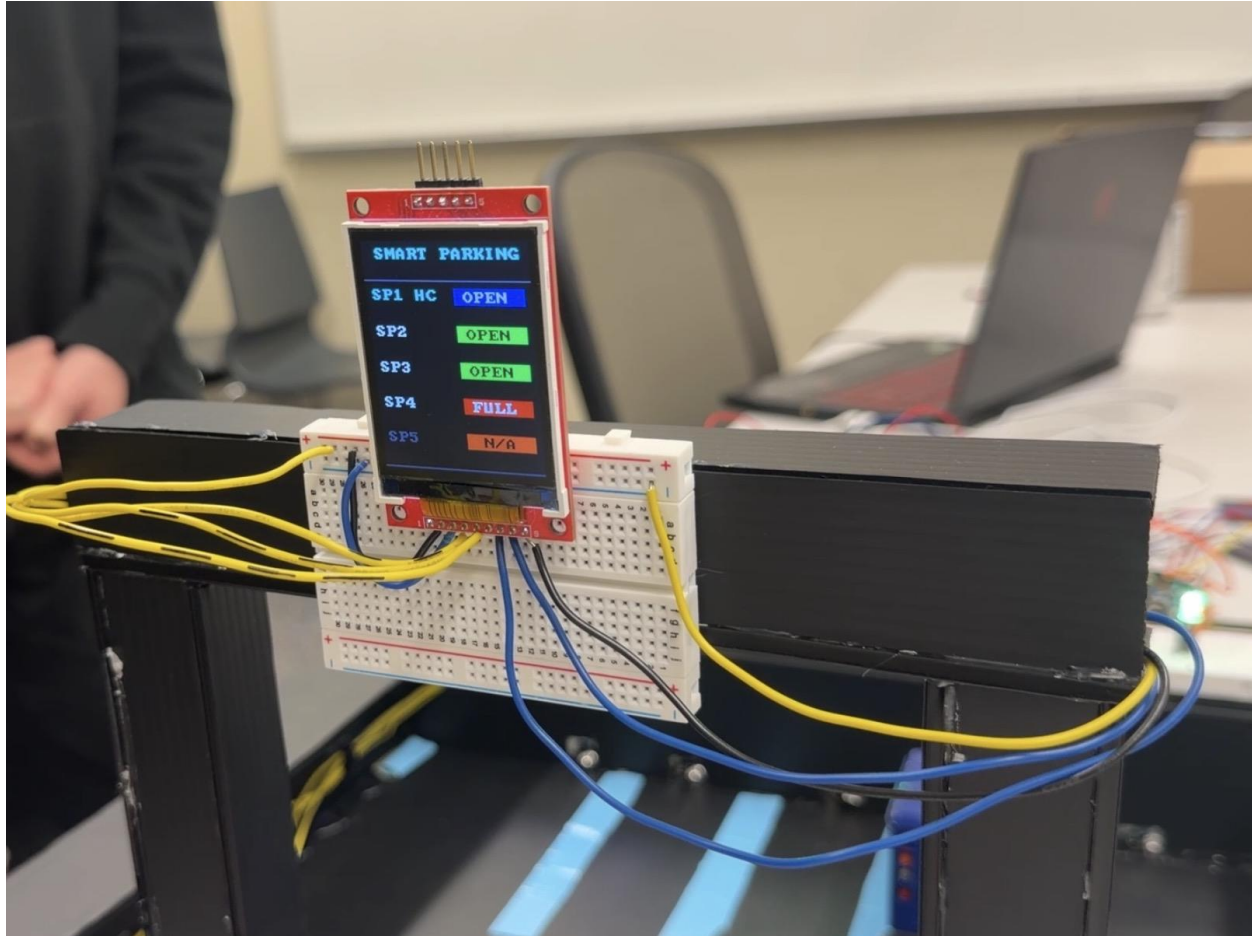


Figure 12 Entrance LCD display

This figure shows the entrance display mounted on the parking model. The LCD provides the user with the status of each parking space, including open, full, handicap, and unavailable or maintenance status.

Testing Procedures, Results, and Analysis

(Nathan Bradshaw – 25%, Fahd Laniyan – 25%, Essa Alansari – 25%, Maitham Sulais – 25%)

The verification framework was based on the Verification Cross Reference Matrix (VCRM), which mapped all twenty project requirements to inspection, demonstration, test, or analysis methods. For the final report, this section was updated from planned testing to

completed final verification. The purpose of testing was to confirm that the Smart Parking Occupancy Detection System works as intended and satisfies the project requirements.

The testing resources included the final parking model, installed sensors, LED indicators, entrance LCD display, Raspberry Pi controller, regulated power hardware, model cars, measuring tools for voltage and continuity checks, and the software needed to observe sensor inputs and display outputs. Testing was completed first at the individual channel level and then at the full system level. This process confirmed that the prototype could detect parking occupancy, update the indicators, calculate available spaces, support special parking states, and operate safely for classroom demonstration.

Physical Prototype, Power, and Safety Testing

The physical prototype was inspected to confirm that the structure supported the parking layout, entrance display area, sensor placement, and internal wiring paths. The corrugated plastic board structure was checked for stability, spacing, and clean organization. The side and lower compartments were also inspected to make sure the wires were routed away from the visible parking surface and were not easily pulled during testing.

The power and wiring layout were checked to support safe classroom demonstration. The system used low voltage power for the controller, sensors, LEDs, and display components. The wiring was kept inside the side compartments as much as possible, and the power path included voltage regulation and fuse protection. This inspection supported the safety intent of UL 62368 1 because the prototype avoided exposed high voltage conductors and reduced the risk of accidental short circuits.

The physical model passed the visual inspection because the parking spaces, entrance display frame, side compartments, and internal cable paths were completed and usable for final integration. The power layout also passed the safety inspection because the prototype operated using low voltage electronics and protected wiring paths.

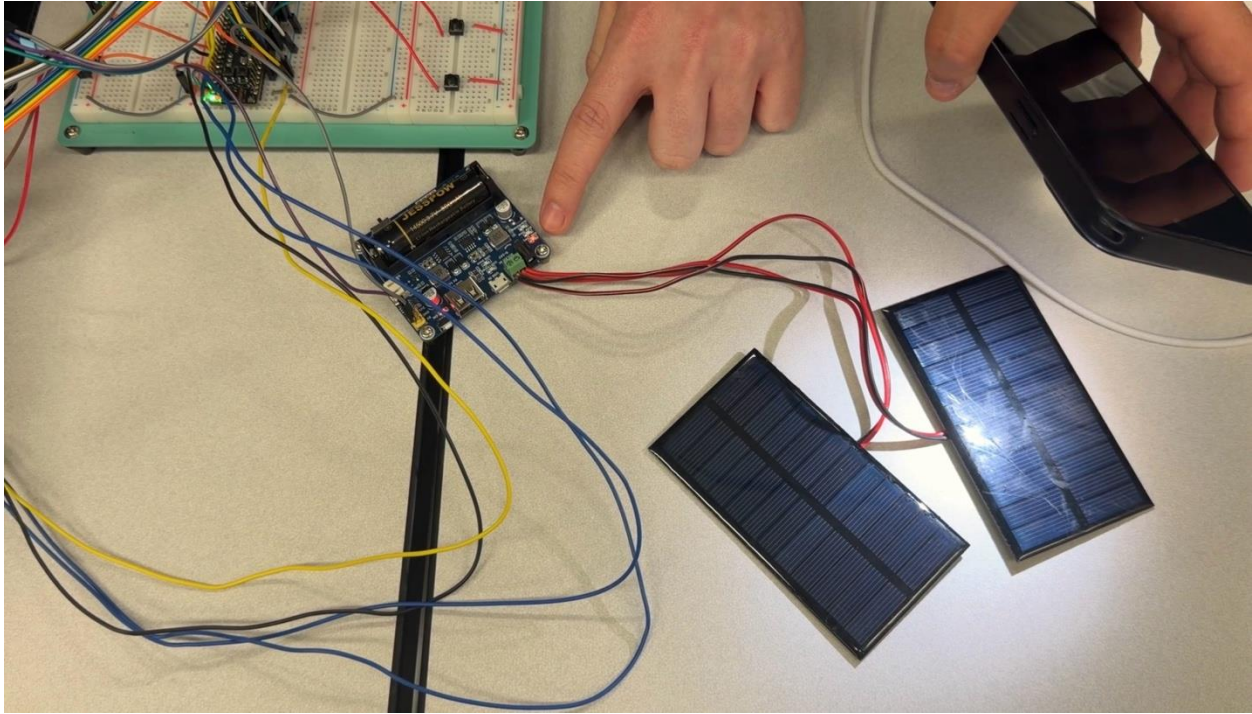


Figure 13 solar and battery module

This figure shows the solar power test setup used to support the renewable energy requirement. The solar panels and battery charging module were connected to verify that the prototype could operate using a solar assisted rechargeable power path.

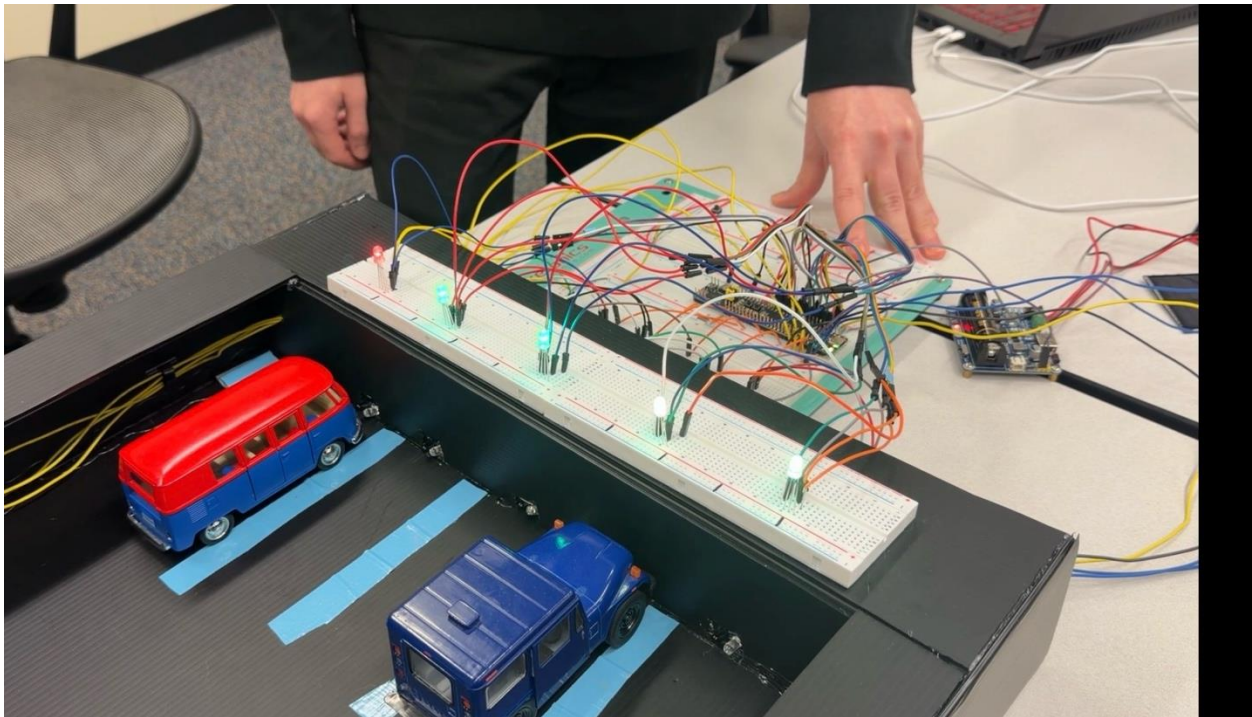


Figure 14 Occupancy Detection

This figure shows the model vehicles placed in parking spaces while the sensor and LED system is active. The LEDs display the parking status and confirm that the system responds to vehicle presence during the final integrated test.

Final System Test Results

Test ID	Requirement checked	Procedure	Expected result	Actual result	Pass or Fail
T1	Req. 1: detect occupied or empty space	Placed a model car in each parking space one at a time.	Sensor detects car presence and clears when removed.	Each sensor detected the model car correctly and returned to empty status when removed.	Pass
T2	Req. 2: control LED indicator	Occupied and cleared each parking space.	LED updates to show the correct parking status.	LED indicators updated correctly for occupied and available spaces.	Pass
T3	Req. 5: calculate available spaces	Tested different numbers of occupied spaces.	System calculates the correct number of available spaces.	Available space count matched the tested parking conditions.	Pass
T4	Req. 9: update within 10 seconds	Placed and removed a car while observing response time.	Display and LED status update within 10 seconds.	System updated parking status within the required 10 second limit.	Pass
T5	Req. 13: handicap indication	Tested the handicap parking space indicator.	Handicap space is shown separately from regular spaces.	Handicap space was shown using the blue indication as planned.	Pass
T6	Req. 14: maintenance mode	Placed a space into maintenance or unavailable status.	System shows a maintenance color.	Maintenance status was shown using the orange indication as planned.	Pass
T7	Req. 17: physical shutdown button	Connected and tested the shutdown button.	System safely responds to the shutdown input.	Shutdown button functioned as required for safe system control.	Pass
T8	Req. 10 and Req. 19: solar assisted rechargeable power	Checked the system power path during operation.	Prototype operates using the solar assisted rechargeable battery system.	System operated correctly with the designed low voltage power setup.	Pass
T9	Req. 15: UL 62368 1 safety intent	Inspected wiring, voltage level, insulation, and protection features.	Prototype avoids exposed high voltage and includes basic protection.	Prototype used low voltage electronics, covered wiring paths, and protection elements.	Pass
T10	Physical structure and cable routing	Inspected the final model, entrance frame, sensor placement, and wiring paths.	Structure supports the demo and keeps wiring organized.	Model, entrance display area, sensor placement, and protected wiring paths passed inspection.	Pass

Table 9 Final verification and testing results by requirement.

Results and Analysis

The final testing results show that the prototype reached the main project goals for the final demonstration. The system successfully detected parking occupancy using the installed sensors and updated the visual indicators as expected. When a model car was placed in a parking space, the system recognized the space as occupied. When the car was removed, the system returned the space to available status. This confirmed that the sensing subsystem worked correctly.

The user interface also worked as intended. The LED indicators showed the correct parking status for each space, and the entrance LCD display provided parking availability information to the user. The special status features were also verified. Blue indication was used for handicap parking spaces, and orange indication was used for maintenance or unavailable spaces. These features helped make the system more clear and useful for drivers.

The system also satisfied the timing requirement. The occupancy status updated within the required 10 second limit, which means the displayed information was fast enough for a driver looking for an available parking space. The available space count also matched the tested parking conditions, showing that the Purdue Proton board processed and aggregated the sensor data correctly.

The power and safety checks showed that the system was suitable for classroom demonstration. The prototype used low voltage electronics, protected wiring paths, voltage regulation, and fuse protection. The wiring was routed through the side and lower compartments to reduce the chance of accidental pulling or short circuits. These results support the safety intent of the project and the UL 62368 1 related requirement.

Overall, all tested requirements passed. The final prototype worked as it should and satisfied the project requirements. Future improvements could include a stronger electronics enclosure, more permanent cable clips, improved weather protection, and a custom PCB to make the wiring cleaner and more durable for repeated use.

Design Challenges

(Essa Alansari – 25%, Maitham Sulais – 25%, Fahd Laniyan – 25%, Nathan Bradshaw – 25%)

Risk management is especially important in a project like this because a simple-looking parking prototype still depends on several tightly linked subsystems. A wiring issue can delay software bring-up, a display decision can force rewiring, and a weak power design can invalidate otherwise good sensing logic. The team therefore needs risks written in a concrete form that ties each event to a likely schedule or design impact, together with both a mitigation step and a contingency response.

Essa Alansari / documentation coordination. IF subsystem updates are not captured in the report and schedule as integration progresses (30%) THEN the final handoff may lose design context and cause rework in later documentation and presentation preparation.
Mitigation: update the report immediately after each major integration meeting and keep

the subsystem narrative aligned with the notebook. Contingency: freeze the wording for completed sections, run a short documentation review meeting, and assign missing sections to specific owners within the same week.

Maitham Sulais / procurement and part availability. IF any required replacement sensor, LED, or power part is unavailable or delayed (35%) THEN wiring and validation may slip because the physical build cannot be completed with like-for-like channels. Mitigation: verify quantities early, keep a prioritized list of critical parts, and identify acceptable substitutes before the stock becomes a problem. Contingency: reduce to the highest-value demonstration subset temporarily while the missing part is replaced, without changing the documented five-space architecture.

Fahd Laniyan / wiring density and schematic execution. IF the real board wiring diverges from the completed schematic or becomes difficult to trace (40%) THEN channel-level debugging time will increase and software integration will be delayed. Mitigation: wire in small verified batches, label channels by space number, and perform continuity checks before closing each wiring segment. Contingency: back out the affected wiring section to the last known-good state and re-implement it using the schematic as the authoritative source.

Nathan Bradshaw / sensor integration and firmware bring-up. IF the controller does not read stable sensor states after hardware hookup (45%) THEN the team may be unable to demonstrate real-time occupancy updates on schedule. Mitigation: bring up one channel first, validate the full sensing-to-LED path, then scale the logic across the remaining spaces. Contingency: preserve a manual-override demonstration path for the interface while isolating the unstable channel and continuing incremental debugging.

ABET Outcomes

(Essa Alansari – 25%, Maitham Sulais – 25%, Fahd Laniyan – 25%, Nathan Bradshaw – 25%)

Maitham Sulais ABET Reflection

Name: Maitham Sulais

Major: Computer Engineering

Course: Senior Design

Design Team: Smart Parking Occupancy Detection System Project

REFLECTIONS:

(a) Describe your personal contributions to the project.

My main contributions to the project were procurement support, documentation support, physical prototype construction, power system planning, and cable organization. I helped select and purchase parts needed for the prototype, and I worked on building the physical structure using corrugated plastic board. I also helped document the design choices, constraints, power management approach, and prototype construction details in the report.

(b) Describe how your contributions to this project built on the knowledge and skills you acquired in earlier course work.

My contributions built on knowledge from earlier electrical and computer engineering courses. The power management part connected to basic circuit concepts such as voltage regulation, current protection, and safe low-voltage operation. The documentation and design explanation also built on previous project and lab work where I had to explain technical decisions clearly and organize engineering results in a formal report.

(c) Describe how you acquired and applied new knowledge as needed. Include learning strategies used.

I had to learn new information during the project, especially about solar charging hardware, rechargeable batteries, fuse protection, and physical cable routing. I used the project requirements, team discussions, online component information, and testing observations to understand how these parts affect safety and reliability. I applied this knowledge by helping organize the power system and making sure the physical structure supported safer wiring and cleaner cable management.

(d) Discuss your ethical and professional responsibilities related to this engineering design experience.

Ethical and professional responsibility was important because the prototype included electrical wiring, battery power, and user-facing indicators. Even though the system was a classroom prototype, it still needed to be safe to handle and clear for users to understand. This meant keeping the system low voltage, protecting wires, organizing the power components, and avoiding design choices that could create safety risks or confuse drivers.

(e) Discuss the impact of the product in economic, environmental, societal, and global contexts.

The project has economic, environmental, societal, and global impact. Economically, a smart parking system can reduce wasted time and improve parking efficiency. Environmentally, the solar-assisted rechargeable battery approach supports cleaner energy use and can reduce dependence on wall power. Societally, the system can make

parking easier for drivers by showing space availability before they enter the lot. In a larger global context, similar systems could support smarter transportation infrastructure and reduce unnecessary traffic caused by drivers searching for parking.

Name: Essa Alansari

Major: Computer Engineering

Course: Senior Design

Design Team: Smart Parking Occupancy Detection System Project

REFLECTIONS:

a) Describe your personal contributions to the project:

My personal contributions were mainly in team leadership, system design, documentation, physical model planning, and final project preparation. At the beginning of the project, I helped the team compare different project ideas and choose the Smart Parking Occupancy Detection System as our final senior design project. After the project was selected, I helped define the project scope, concept of operations, system requirements, and general design direction.

I was selected as the team leader. My role was to help keep the team organized and make sure everyone understood the project goals. I worked with Maitham on system design and demo planning, while Fahd and Nathan focused more on the hardware and software work. I also helped with the system requirements and the VCRM. I prepared to explain the requirement allocation and verification strategy to the instructor.

I also helped with the physical design of the parking model. I worked with Maitham on planning and building the model, including the parking layout, sensor placement, entrance display location, and cable management. During the project, we made design decisions such as using the parking map with LED indicators and organizing the wiring inside the structure. Near the final stage, I worked on the final report and poster preparation. Overall, my main role was to help organize the project, support design decisions, document the work, and make sure the final prototype matched the project requirements.

b) Describe how your contributions to this project built on the knowledge and skills you acquired in earlier coursework:

My work on this project built on skills I learned in computer engineering courses. These include embedded systems, microprocessor systems, circuit design, and technical

writing. From earlier courses, I learned how a system can be divided into smaller subsystems, such as sensing, control, power, and user interface. This helped me understand how to organize the project and explain it clearly in the report.

My background in embedded systems also helped me understand how sensors, GPIO pins, LEDs, and a central controller work together. Even though I was not the main software developer, I was still able to understand how the Raspberry Pi reads sensor data and updates the LED indicators and entrance display. This helped me communicate better with the hardware and software team members.

I also used my technical writing skills. The final report needed clear sections for motivation, background, requirements, implementation, testing, and future work. Since I helped with documentation, I used these skills to explain the project in a clear and professional way. My coursework also helped me understand why safety, standards, and verification are important in engineering design.

c) Describe how you acquired and applied new knowledge as needed to contribute to this project. What learning strategies did you use?

During this project, I learned more about how a senior design project moves from an idea to a working prototype. I learned how to connect project requirements to real design choices. I also learned more about the VCRM, testing plans, smart parking systems, sensor placement, solar power, and cable management.

The main learning strategies I used were team discussion, reviewing class materials, looking at sample reports, and using instructor feedback. For example, our team discussed whether to use an LCD display or a parking map with LED indicators at the entrance. After discussion, we decided that the parking map with LED indicators would be easier for users to understand quickly.

I also learned through trial and improvement. The physical model changed as we worked on it because we needed enough space for sensors, wiring, and the entrance display. When the design changed, I helped update the report and poster so they matched the real final prototype. This helped me understand that engineering design is not only about the first idea. It is also about improving the design as new problems appear.

d) Discuss your ethical and professional responsibilities as they relate to this engineering design experience.

My ethical and professional responsibilities included supporting a safe design, communicating clearly with my team, and being honest in the documentation. Since our project includes batteries, solar charging, wiring, sensors, and user facing hardware, safety

was very important. We included requirements for low voltage operation, battery voltage monitoring, overcharge protection, insulation, current limiting, and UL 62368 1 safety intent.

As team leader, I also had a responsibility to help the team stay organized. I needed to help coordinate tasks, support communication, and make sure the report, poster, testing, and design work were moving forward. I also had to make sure each team member's work was represented fairly.

Another responsibility was to make the system useful and clear for users. The goal of the project is to help drivers understand parking availability quickly. Adding special colors for handicap spaces and maintenance spaces made the system more clear and helpful. This is important because an engineering design should not only work technically, but it should also be easy and safe for people to use.

e) Consider what the impact of the product could have in economic, environmental, societal, and global contexts.

Economically, this smart parking system could help drivers save time by showing available parking spaces before they enter the lot. In a larger system, it could reduce traffic inside parking areas and help campuses, malls, or businesses manage parking better. Since the prototype uses low cost sensors, LEDs, and a Raspberry Pi, the design can be affordable and practical.

Environmentally, the system could reduce extra driving inside parking lots. If drivers find spaces faster, they may use less fuel and produce fewer emissions. The project also uses solar power and rechargeable batteries, which supports more sustainable operation.

Societally, the system can make parking easier and less stressful for drivers. It also improves accessibility by showing handicap spaces separately. The maintenance color also helps users know when a space should not be used. These features make the system more helpful and easier to understand.

Globally, parking problems happen in many places, including cities, universities, malls, airports, and public facilities. A larger version of this system could be used in different countries and different types of parking lots. Our project is a small prototype, but the idea could be expanded later with wireless sensors, a web dashboard, data logging, and outdoor weather protection.

Name: Fahd Laniyan

Major: Computer Engineering

Course: Senior Design

Design Team: Smart Parking Occupancy Detection System Project

REFLECTIONS

(a) Describe your personal contributions to the project.

My primary role on the team was hardware development lead. I was responsible for the KiCad schematic, GPIO allocation, board wiring, LED and sensor wiring preparation, and electrical integration of the full prototype. At the start of the project I developed the system block diagram, which established how the sensing, control, user interface, and power subsystems connect to each other. This diagram became the foundation for the requirements and design work that followed.

After the block diagram was completed, I created the KiCad schematic that documents the full electrical design. This included assigning GPIO pins on the Proton board for all five IR sensor outputs, fifteen LED channels across five RGB indicators, five manual-override pushbuttons, the combined shutdown and mode-reset button, and the battery voltage sensing circuit. I selected current-limiting resistor values for each LED channel based on the supply voltage and LED forward voltage specifications, and I verified the allocation to confirm that no pin was double-assigned.

During the physical build phase, I led the sensor mounting and wiring work together with Nathan. I installed the IR sensor modules at each parking space, routed the sensor and LED wiring through the side and lower compartments of the model, and connected all channels to the Proton board according to the schematic. I also installed the entrance LCD display using extended connections and performed continuity checks on each channel before software integration began. My contributions at this stage were essential for making software testing possible.

(b) Describe how your contributions to this project built on the knowledge and skills you acquired in earlier coursework.

My hardware work on this project built directly on skills from embedded systems courses, circuit design coursework, and laboratory work with microcontrollers. In earlier courses I learned how GPIO pins are allocated on microcontroller development boards, how to

select current-limiting resistors for LED circuits, and how to read digital sensor outputs without additional signal conditioning. These skills were applied directly when I designed the schematic and selected component values for the RGB LED channels and IR sensor interfaces. The IR obstacle sensor was selected specifically because it produced a clean digital output the Proton could read directly on a GPIO pin, avoiding the timed pulse measurement and noise susceptibility that made ultrasonic sensors a worse fit for this application.

My experience with KiCad from earlier PCB design work also helped me complete the schematic efficiently. I already understood the process of placing components, assigning reference designators, routing nets, and checking for errors before finalizing a design. The schematic for this project was more complex than earlier assignments because it covered multiple subsystems on one sheet, but the underlying process was the same.

Technical writing and documentation skills from earlier courses helped me write the hardware design sections of the report clearly. I was able to explain the GPIO allocation, resistor selection, sensor topology, and wiring strategy in language that connected the schematic decisions to the system requirements and physical implementation.

(c) Describe how you acquired and applied new knowledge as needed to contribute to this project. What learning strategies did you use?

Several aspects of this project required me to learn new material. I had not previously worked with the Waveshare Solar Power Manager module or the MPPT charging approach, so I studied the module datasheet to understand the configuration jumpers, output voltage settings, and current limits before integrating it into the schematic. This directly affected how I connected the solar input rail, the battery path, and the regulated output to the rest of the power architecture.

I also learned how to plan wiring for a physical prototype where conductors need to be routed through enclosed compartments and where labeling and organization matter as much as the electrical connections themselves. My main learning strategies were reading component datasheets, reviewing the schematic against the physical layout before making connections, and performing incremental continuity checks after completing each wiring segment.

I also cross-checked my GPIO allocation with Nathan before finalizing the schematic to make sure the pin assignments matched what the firmware required. This caught a

potential mismatch before wiring was finalized. When problems appeared during integration, I used the schematic as the authoritative reference to trace wiring back to the source of an issue rather than guessing. This approach helped resolve channel-level problems quickly and reinforced the habit of treating the schematic as a live document that must match the physical build.

(d) Discuss your ethical and professional responsibilities as they relate to this engineering design experience.

As the hardware lead, my most direct ethical responsibility was to produce an electrically safe design. The prototype operates in a classroom setting where it is handled by team members and inspected by the instructor and other students. I had a responsibility to ensure that no accessible conductor exceeded safe voltage levels, that wiring was protected from accidental contact, and that the battery and charging path included the protection elements required by the UL 62368-1 safety intent in the project requirements. I addressed this by keeping all logic and peripheral power on the 3.3V rail, including fuse protection in the power path, and routing wiring inside enclosed compartments rather than leaving it exposed on the parking surface.

I also had a professional responsibility to produce a schematic that accurately represented the physical build. If the schematic and the actual wiring diverged, it would make troubleshooting harder for the team and would reduce the reliability of any documentation submitted with the project. I treated the schematic as a binding design document and updated it whenever a design decision changed during construction.

Finally, because my hardware wiring directly affected Nathan's software integration, I had a responsibility to communicate GPIO assignments, signal behavior, and any wiring changes clearly and promptly. Keeping that communication open prevented hardware and software mismatches and made the integration process more organized for the whole team.

(e) Consider what the impact of the product could have in economic, environmental, societal, and global contexts.

Economically, the sensing and wiring approach selected for this project keeps the per-space cost low. IR obstacle sensor modules and standard through-hole RGB LEDs are inexpensive and widely available, which means the design could be scaled to larger parking

facilities without a large increase in cost per space. However, the point-to-point wiring approach is only cost-effective at the five-space prototype scale. A production deployment would require a bus-based or multiplexed architecture to manage wire count and installation labor, which changes the economic analysis significantly and would need to be addressed before the design could be deployed competitively.

Environmentally, a reliable occupancy detection system reduces the time drivers spend searching for parking, which lowers fuel consumption and vehicle emissions inside parking areas. The hardware design also supports the solar-assisted power approach used in this project, meaning the sensing and indication hardware draws from renewable energy during operation rather than continuous wall power.

Societally, clear and accurate occupancy indication improves the experience for all parking users, including drivers with disabilities who benefit from knowing the location and availability of designated accessible spaces before entering the lot. The hardware design supports this through the RGB LED capability, which allows distinct color coding for regular, handicap, and maintenance-status spaces from a single physical indicator per space.

Globally, the hardware architecture used in this project is simple enough to be replicated in a wide range of settings. The components are available internationally, the design does not depend on cloud infrastructure, and the low-voltage safety approach is consistent with widely adopted electrical safety standards. A more permanent version of this system could be deployed in parking facilities in different countries and climates with appropriate weatherproofing and enclosure improvements.

Name: Nathan Bradshaw

Major: Electrical Engineering

Course: Senior Design

Design Team: Smart Parking Occupancy Detection System Project

REFLECTIONS:

(a) Describe your personal contributions to the project.

My primary contribution was designing and implementing the complete embedded firmware for the Smart Parking System on the Purdue Proton board in C. This included IR occupancy sensing, an RGB LED state machine, a custom ILI9341 LCD

driver over SPI, button handling with debounce and hold detection, and a serial analytics subsystem. I designed and implemented the main polling loop, the three tier space classification system, the operator override mechanism, and a recommendation algorithm that suggests the best available space based on runtime occupancy history. I also coordinated with teammates to ensure the firmware aligned with the hardware subsystems developed in parallel.

(b) Describe how your contributions to this project built on the knowledge and skills you acquired in earlier course work.

My embedded systems coursework provided the foundation for GPIO configuration, active-low sensor logic, and microcontroller interfacing. Digital circuits coursework informed my software debounce implementation, and my communication protocols coursework gave me the framework for SPI transactions used in the LCD driver. C programming coursework enabled me to manage per space state variables cleanly and produce formatted output to both the serial console and the display efficiently.

(c) Describe how you acquired and applied new knowledge as needed to contribute to this project. What learning strategies (see definition on first page) did you employ to do so?

The largest knowledge gap I had to close was driving the ILI9341 LCD without a pre-written library. I worked directly from the datasheet and Pico SDK documentation, implementing and verifying each routine incrementally before building the next. I also relied on peer collaboration when resolving integration issues between my firmware and the team's hardware subsystems. My primary learning strategies were targeted documentation research, interface decomposition into small testable units, and iterative testing to validate each layer before moving forward.

(d) Discuss your ethical and professional responsibilities as they relate to this engineering design experience.

A central ethical responsibility in this project was the correct handling of handicap designated spaces, which carry legal obligations under the ADA. I designed the button logic to require a deliberate hold press before any space designation could be changed, reducing the risk of accidental reclassification.

(e) Consider what the impact of the product of this engineering design experience could have in economic, environmental, societal, and global contexts. Discuss how you would make (or did make) an informed judgment (see definition on first page) as to your product's impact in each of these four contexts.

Economically, the system reduces unproductive fuel consumption and search time for drivers while keeping deployment costs accessible through low cost hardware like the Raspberry Pi Pico. Environmentally, reducing search driven vehicle circulation lowers localized emissions, though the manufacturing and disposal

footprint of the electronic components is a cost that responsible deployment must account for. Societally, the system's clearest benefit is improving parking equity and giving all users real time availability information. Globally, the operator override and Unavailable space designation reflect design decisions made with real world deployment in mind, where purely sensor driven logic is insufficient. I weighed these factors by considering scalability, maintenance requirements, and long term reliability.

Conclusions and Future Work

(Essa Alansari – 100%)

The strongest conclusion in the final report is that the Smart Parking Occupancy Detection System was successfully developed from an initial idea into a working prototype. Over the course of the project, the team moved through concept development, requirement definition, VCRM completion, component selection, physical construction, hardware integration, and testing. By the end of the project, the team completed the parking model, mounted the sensors, installed the entrance LCD display, connected the LED indicators, and integrated the system so that parking availability could be detected and shown to the user in real time.

The final prototype demonstrates the main goals of the project. The system monitors five parking spaces using IR sensors, processes the sensor inputs through a Raspberry Pi, and updates the parking status through visual indicators. The LED indicators provide space level feedback, while the entrance LCD gives users a clear view of parking availability before entering the lot. The project also includes added features such as handicap space indication and maintenance status indication, which improve usability and make the system more practical. In addition, the use of solar assisted power and rechargeable battery storage supports the project goal of creating an environmentally friendly and low voltage embedded system.

Another important conclusion is that the project showed the value of system level planning and teamwork. The requirements and VCRM helped organize the design and verification process, while regular meetings helped the team coordinate mechanical, electrical, software, and documentation tasks. The project also showed that even a small prototype requires strong integration between sensing, processing, user interface, and power management. Challenges such as wiring, sensor placement, and interface decisions were solved through discussion, testing, and design updates.

Future work for this project would focus on making the system more advanced and more scalable. One possible improvement is to expand the prototype beyond five parking spaces so it can represent a larger parking lot. Another improvement would be to add

wireless communication or a web based dashboard so parking information could be viewed remotely. The system could also be improved by adding stronger enclosure protection, better cable organization, and a more permanent hardware platform such as a custom PCB instead of prototype wiring. Additional future work could include more detailed occupancy history logging, improved filtering for false detections, and better outdoor readiness for real world parking environments.

Overall, the project achieved its main purpose by creating a functional smart parking prototype that can detect parking occupancy, communicate parking status clearly, and demonstrate the value of embedded systems in solving a practical everyday problem.

REFERENCES (NUMERICAL)

[2] ECE 49200 – Project Report Template, Purdue University in Indianapolis, Fall 2025. Available: <https://brightspace.iu.edu>

[3] ECE 49200 Grading Rubric, Purdue University in Indianapolis, Spring 2026. Available: <https://brightspace.iu.edu>

[4] Risk and Misc, ECE 49200 course materials, Purdue University in Indianapolis, Spring 2026. Available: <https://brightspace.iu.edu>

[5] Introduction, ECE 49200 course materials, Purdue University in Indianapolis, Spring 2026. Available: <https://brightspace.iu.edu>

[6] UL 62368-1, Audio/Video, Information and Communication Technology Equipment – Part 1: Safety Requirements, 3rd ed., Underwriters Laboratories, 2019. Available: <https://www.ul.com/resources/ul-62368-1>

- [7] Purdue Proton Board, ECE 362 Microprocessor Systems and Interfacing, Purdue University in Indianapolis. Available: <https://ece362-purdue.github.io/proton-labs/>
- [8] V. Paidi, H. Fleyeh, J. Häkkinen, and R. G. Nyberg, "Smart parking sensors, technologies and applications for open parking lots: a review," *IET Intelligent Transport Systems*, vol. 12, no. 8, pp. 735–741, 2018. Available: <https://doi.org/10.1049/iet-its.2017.0406>
- [9] H. M. Marhoon et al., "Design and Implementation of an RFID-Based Smart Parking Control System with Infrared Sensors and Queueing-Theory Traffic Modelling," *Control Systems and Optimization Letters*, vol. 3, no. 1, 2025. Available: <https://doi.org/10.59247/csol.v3i1.261>
- [10] L. Baroffio et al., "A visual sensor network for parking lot occupancy detection in Smart Cities," in *Proc. IEEE 2nd World Forum on Internet of Things (WF-IoT)*, Milan, Italy, 2015, pp. 745–750. Available: <https://doi.org/10.1109/WF-IoT.2015.7389147>

APPENDIX A. SPECIFICATION OF ALL HARDWARE PARTS

Table A1 records the current hardware architecture for the five-space prototype. Specific manufacturer part numbers should be added in the final report once the build list is locked.

Item	Current quantity	Function in system	Status / notes
IR occupancy sensors	5	Detect vehicle presence at each monitored space.	U2–U6 in schematic. Outputs IR_SLOT1_OUT–IR_SLOT5_OUT to Proton GPIO pins. Operates on +3V3 rail.
RGB LED indicators (D1–D5) with 330 Ω and 33 Ω resistors	5	Show local occupancy state (green = free, red = occupied, yellow = maintenance) at each space. Also mirrored on entrance map.	D1–D5 in schematic. 5 $\times$ 330 Ω resistors, 10 $\times$ 33 Ω resistors (R1–R15), one per color channel.
Entrance map LEDs	5	Mirror space status on the entrance parking map.	Selected on March 9 as the lot-level interface.
Purdue Proton board (ARM Cortex-M33)	1	Reads all five IR sensor channels, maintains occupancy state, drives RGB LEDs and entrance map, and handles override and shutdown inputs.	Provided by ECE lab. Selected for its 48 GPIO pins, ADC channels, and ARM Cortex-M33 core, which cover all sensing, indication, and override I/O requirements.
Manual override pushbuttons	5	Allow authorized manual change of status when required.	Scaled to current five-space scope.

Shutdown button	1	Safely powers down the controller and interface subsystem.	Required by system requirements.
Solar charging module	1	Manages solar input into the rechargeable power path.	Part of documented power subsystem.
Rechargeable battery pack	1	Stores energy for continued operation.	Protected low-voltage storage path.
5 V regulator	1	Provides regulated supply to logic and interface hardware.	Included in schematic-level power plan.
Voltage-sensing circuit	1	Monitors battery voltage for safe charging behavior.	Required for overcharge prevention.
Fuse / protection hardware	1 set	Provides current limiting and protection.	Supports UL 62368-1-related safety intent.
Prototype board / parking model	1	Houses spaces, wiring channels, and subsystem integration.	Physical model already constructed.
Breadboard and hookup wiring	as needed	Supports prototyping and electrical interconnection.	Board wiring is currently in progress.

Table 10 Current hardware parts and functions.

APPENDIX B. SPECIFICATION OF ALL SOFTWARE AND TOOLS USED

Tool / artifact	Role in project	status
Team notebook	Records decisions, requirements, progress, and meeting actions.	In use
VCRM spreadsheet	Tracks requirements, verification methods, and subsystem mapping.	Completed
KiCad 9.0 schematic capture	Documents all wiring, GPIO allocation, power rail architecture, sensor and LED hookup, and protection elements. Schematic file: Smart_parking_project_.kicad_sch.	Schematics completed
Proton board firmware development (C / embedded)	Implements GPIO-based IR sensor polling, debounce filtering, RGB LED state control, ADC battery monitoring, override and shutdown handling. Targets ARM Cortex-M33 on Proton board.	Complete
Report template and course formatting guidance	Provides the required report structure and Purdue formatting conventions.	In use

Table 11 Current software and development tools.

APPENDIX C. LIST OF ABBREVIATIONS

CONOPS: Concept of operations

GPIO: General-purpose input/output

IR: Infrared

LED: Light-emitting diode

UL: Underwriters Laboratories

VCRM: Verification Cross Reference Matrix

ADC: Analog-to-digital converter

MCU: Microcontroller unit

RGB: Red-green-blue (multi-color LED)

BOM: Bill of materials

APPENDIX D. BILL OF MATERIALS

Table D1 lists all purchased components for the five-space prototype with confirmed costs from the team's purchase request. The Purdue Proton board is provided by the ECE lab and carries no direct purchase cost. Total procurement cost is \$199.61.

Item	Qty	Unit cost	Extended cost	Comments
IR sensor	5	\$8.79	\$8.79	IR obstacle sensor module pack. One module per monitored space (U2–U6 in schematic).
Space status LED	5	\$8.99	\$8.99	One per monitored space.
Entrance map LED	5	\$7.99	\$7.99	7 RGB LED indicator pack (D1–D5 space LEDs; additional LEDs for entrance map). 330 Ω and 33 Ω resistors on each channel per schematic (R1–R15).
Purdue Proton board (ARM Cortex-M33)				
Manual override pushbutton	5	\$13.60	\$13.60	8-pushbutton pack. BTN1–BTN5 = manual override (one per space); BTN_MODE_RESET = shutdown/mode.
Shutdown pushbutton	1	\$13.99	\$13.99	Included in 8-button pack above (BTN_MODE_RESET in schematic).
Solar charging hardware	1	\$7.84	\$7.84	Solar charging manager kit (U1 in schematic).
Rechargeable battery pack	1	\$5.89	\$5.89	Rechargeable lithium-ion battery pack.

5 V regulator	1	\$9.96	\$9.96	5 V regulator. Supplies logic rail to Proton board and peripherals.
Voltage-sensing components	1 set	\$35.99	\$35.99	Voltage sensing circuit module (J2 in schematic). Monitors BATT_POS for overcharge protection.
Fuse / protection parts	1 set	\$23.78	\$23.78	Insulation, fuse, and protective components for UL 62368-1 compliance intent.
Prototype board materials	1 set	\$35.99	\$35.99	Juvale corrugated plastic board. Physical parking model structure.
Breadboard and wiring	1 set	\$23.78	\$23.78	Breadboards (\$7.89) + hookup wires (\$15.89).

Table 12 Bill of materials for the five-space prototype.

Requisition total: \$199.61. Purdue Proton board provided by ECE lab (\$0.00 direct cost). Additional items purchased separately: resistor pack 220–330 Ω (\$9.99), NPN transistors (\$7.99), pin headers (\$14.99), model cars for demonstration (\$9.99), solar panel (\$19.83).

APPENDIX E. VERIFICATION CROSS REFERENCE MATRIX

Table E1 is the Verification Cross Reference Matrix for all twenty system requirements. It includes the full requirement text with the required “shall” wording, level, type, verification method, rationale, source, and subsystem allocation, as established at the February 11 VCRM meeting.

Req.	Requirement (Threshold)	Level	Type	Method	Rationale	Source	Subsystem Allocation
1	The Edge Sensor Node Subsystem shall detect whether its assigned parking space is occupied or empty.	Threshold	Functional	Test	The core function of the system is real-time occupancy detection. Without accurate per-space detection the system cannot report availability or drive any indicators.	CONOPS	Edge Sensor Node Subsystem
2	The Edge Sensor Node Subsystem shall control an LED indicator to represent the occupancy status of its assigned parking space.	Threshold	Functional	Demonstration	Drivers require immediate visual feedback at each space. A per-space RGB LED indicator communicates status without requiring any display or connectivity infrastructure.	CONOPS	Edge Sensor Node Subsystem
3	The Communication Subsystem	Threshold	Functional	Demonstration	The central controller must receive status	System Block Diagram	Communication Subsystem

	shall transmit parking availability data from each Edge Sensor Node to the Central Processing Subsystem.				from all five spaces to compute aggregate availability and update the entrance map.		
4	The Central Processing Subsystem shall store historical parking occupancy data for analysis purposes.	Threshold	Functional	Test	Occupancy history enables future analysis of usage patterns and demonstrates data logging capability beyond simple real-time indication.	CONOPS	Central Processing Subsystem
5	The Central Processing Subsystem shall calculate the total number of available parking spaces.	Threshold	Performance	Test	Drivers need a quick count of available spaces before entering the lot. The controller must aggregate all five sensor states into a single available-space count.	CONOPS	Central Processing Subsystem
6	The Edge Sensor Node Subsystem shall interface with a proximity sensor installed	Threshold	Functional	Inspection / Test	A dedicated sensor per space achieves independent channel-level detection.	System Block Diagram	Edge Sensor Node Subsystem

	at each individual parking space.				Shared sensing would reduce reliability and complicate troubleshooting.		
7	The Central Processing Subsystem shall use the Purdue Proton board (ARM Cortex-M33) as the main processing unit.	Threshold	Physical	Inspection	A single central controller simplifies architecture, reduces wiring complexity, and ensures all sensing, logic, and output decisions share one consistent state table.	System Block Diagram	Central Processing Subsystem
8	The Central Processing Subsystem shall process and aggregate occupancy data received from all Edge Sensor Nodes.	Threshold	Functional	Demonstration	Local aggregation avoids dependence on any external network or cloud service and keeps the system self-contained for classroom demonstration.	CONOPS	Central Processing Subsystem
9	The Edge Sensor Node Subsystem shall update occupancy status within 10 seconds of	Threshold	Performance	Test	A 10-second maximum update latency ensures the displayed status remains useful for drivers	CONOPS	Edge Sensor Node Subsystem

	detecting a change.				actively looking for parking. Longer delays would make the system unreliable for real-time guidance.		
10	The Power Management Subsystem shall operate using energy supplied by a solar charging system.	Threshold	Functional	Demonstration	Solar power enables continuous operation without a wall outlet, supporting the portability and sustainability goals documented in the CONOPS.	CONOPS	Power Management Subsystem
11	The Power Management Subsystem shall ensure that no accessible conductive parts exceed 30V DC under normal operating conditions.	Threshold	Safety	Test / Analysis	Low-voltage operation below 30V DC keeps the prototype safe for classroom handling. This boundary is consistent with the low-voltage safety intent of UL 62368-1.	Safety Constraint / UL 62368-1	Power Management Subsystem
12	The Power Management Subsystem shall prevent	Threshold	Safety	Test	Lithium-ion battery overcharge is a known fire and	Safety Constraint	Power Management Subsystem

	battery overcharge by continuously monitoring battery voltage using a voltage sensing circuit.				safety risk. Continuous voltage monitoring via the sensing circuit allows the controller to detect and respond to overcharge conditions.		
13	The Central Processing Subsystem shall identify designated handicap parking spaces separately from regular spaces in the display interface.	Threshold	Functional	Demonstration	Handicap space identification was added at the January 23 class meeting as a required accessibility feature. Separate indication ensures usability for all drivers.	Use Case / January 23 Class Meeting	Central Processing Subsystem
14	The Edge Sensor Node Subsystem shall support a maintenance status mode that displays yellow or orange LED indication during construction or servicing.	Threshold	Functional	Demonstration	A maintenance mode prevents drivers from attempting to park in a space under service. Yellow or orange indication is a widely recognized caution signal.	Use Case	Edge Sensor Node Subsystem

15	The Electrical Design Subsystem shall comply with UL 62368-1 by incorporating proper insulation, current limiting, and protection against electric shock and fire hazards for low-voltage electronic equipment.	Threshold	Safety	Inspection / Analysis	UL 62368-1 is the applicable safety standard for low-voltage electronic equipment. Compliance ensures the prototype meets recognized safety intent for insulation, current limiting, and shock/fire protection.	UL 62368-1 Safety Standard	Electrical Design Subsystem
16	The Power Management Subsystem shall support simultaneous battery charging and normal system operation.	Threshold	Functional	Test	The system must remain operational while the solar charging path replenishes the battery. Interrupting operation during charging would make the system unusable in continuous deployment.	CONOPS	Power Management Subsystem
17	The User Interface Subsystem shall include a physical shutdown	Threshold	Safety / Functional	Demonstration	A physical shutdown button enables safe controlled power-down without abrupt	Use Case / CONOPS	User Interface Subsystem

	button that safely powers off the Central Processing Subsystem.				disconnection, protecting the controller and file system from data corruption.		
18	The Edge Sensor Node Subsystem shall include a manual override input to allow authorized personnel to change the parking status indication when required.	Threshold	Functional	Demonstration	Manual override allows operators to correct sensor errors or reserve spaces without modifying firmware, supporting reliable real-world operation independent of sensor accuracy.	Use Case	Edge Sensor Node Subsystem
19	The Power Management Subsystem shall store energy using rechargeable lithium-ion or equivalent rechargeable battery technology.	Threshold	Physical	Inspection	Rechargeable battery storage enables continuous operation between solar charging cycles. Lithium-ion technology provides sufficient energy density for the prototype power budget.	CONOPS	Power Management Subsystem
20	The Edge Sensor Node Subsystem	Threshold	Performance	Test / Analysis	Without debounce filtering, brief	CONOPS	Edge Sensor Node Subsystem

	shall perform local occupancy validation filtering to reduce false detections caused by environmental noise or transient objects.				sensor transients would generate visible flicker and incorrect availability counts, reducing system credibility.		
--	---	--	--	--	--	--	--

Table 13 Verification cross reference matrix.

APPENDIX F. PROOF OF HARDWARE RETURN

Hardware name	Purchased or donated	Who (school/sponsor/student) purchased/donated	Check if returned	Comment
IR sensor	Purchased	purchased		
Space status LED	Purchased	purchased		
Entrance map LED	Purchased	purchased		
Purdue Proton board (ARM Cortex-M33)	Purchased	purchased		
Manual override pushbutton	Purchased	purchased		
Shutdown pushbutton	Purchased	purchased		
Solar charging hardware	Purchased	purchased		
Rechargeable battery pack	Purchased	purchased		
5 V regulator	Purchased	purchased		
Voltage-sensing components	Purchased	purchased		
Fuse / protection parts	Purchased	purchased		
Prototype board materials	Purchased	purchased		
Breadboard and wiring	Purchased	purchased		

Table 14 Proof of Hardware Return.

APPENDIX G. CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "pico/stdlib.h"
#include "hardware/gpio.h"
#include "hardware/spi.h"

#define IR_SPACE_1    27
#define IR_SPACE_2    29
#define IR_SPACE_3    34
#define IR_SPACE_4    39
#define IR_SPACE_5    44

#define BTN_SPACE_1   17
#define BTN_SPACE_2   18
#define BTN_SPACE_3   19
#define BTN_SPACE_4   20
#define BTN_SPACE_5   21

#define LED1_R  14
#define LED1_G  15
#define LED1_B  16
#define LED2_R  11
#define LED2_G  12
#define LED2_B  13
#define LED3_R   8
#define LED3_G   9
#define LED3_B  10
#define LED4_R   5
#define LED4_G   6
#define LED4_B   7
#define LED5_R   2
#define LED5_G   3
#define LED5_B   4

#define LCD_SPI      spi1
#define LCD_MOSI     47
#define LCD_SCK      46
#define LCD_CS       45
#define LCD_DC       43
#define LCD_RESET    42
#define LCD_LED      41

#define ILI9341_SWRESET 0x01
```



```

#define ILI9341_SLPOUT      0x11
#define ILI9341_DISPON      0x29
#define ILI9341_CASET       0x2A
#define ILI9341_PASET       0x2B
#define ILI9341_RAMWR       0x2C
#define ILI9341_MADCTL      0x36
#define ILI9341_PIXFMT      0x3A

#define COLOR_BLACK         0x0000
#define COLOR_RED           0xF800
#define COLOR_GREEN         0x07E0
#define COLOR_BLUE          0x001F
#define COLOR_CYAN          0x07FF
#define COLOR_YELLOW        0xFFE0
#define COLOR_WHITE         0xFFFF
#define COLOR_GRAY          0x8410
#define COLOR_ORANGE        0xFC00

#define LCD_WIDTH           240
#define LCD_HEIGHT          320

#define SPACE_NORMAL        0
#define SPACE_HANDICAP      1
#define SPACE_UNAVAILABLE   2

#define HOLD_THRESHOLD_MS   1500
#define DEBOUNCE_MS         50
#define NUM_SPACES          5

const uint8_t sensor_pins[NUM_SPACES] = {IR_SPACE_1, IR_SPACE_2,
IR_SPACE_3, IR_SPACE_4, IR_SPACE_5};
const uint8_t button_pins[NUM_SPACES] = {BTN_SPACE_1, BTN_SPACE_2,
BTN_SPACE_3, BTN_SPACE_4, BTN_SPACE_5};

const uint8_t button_controls_space[NUM_SPACES] = {0, 1, 2, 3, 4};

const uint8_t led_pins[NUM_SPACES][3] = {
    {LED1_R, LED1_G, LED1_B}, {LED2_R, LED2_G, LED2_B},
    {LED3_R, LED3_G, LED3_B}, {LED4_R, LED4_G, LED4_B},
    {LED5_R, LED5_G, LED5_B}
};

uint8_t space_type[NUM_SPACES] = {SPACE_HANDICAP, SPACE_NORMAL,
SPACE_NORMAL, SPACE_NORMAL, SPACE_UNAVAILABLE};

```

```

bool override_active[NUM_SPACES]    = {false};
bool override_occupied[NUM_SPACES] = {false};

bool      btn_was_pressed[NUM_SPACES] = {false};
uint32_t btn_press_time[NUM_SPACES]  = {0};
bool      btn_hold_fired[NUM_SPACES]  = {false};

bool last_state[NUM_SPACES] = {false};
uint8_t saved_type[NUM_SPACES] = {SPACE_HANDICAP, SPACE_NORMAL,
SPACE_NORMAL, SPACE_NORMAL, SPACE_UNAVAILABLE};

absolute_time_t occupation_start_time[NUM_SPACES];
uint32_t total_occupied_ms[NUM_SPACES] = {0};
uint32_t occupation_count[NUM_SPACES] = {0};
uint32_t last_print_time = 0;

const uint8_t font8x8[][8] = {
    {0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00},
    {0x18,0x3C,0x3C,0x18,0x18,0x00,0x18,0x00},
    {0x36,0x36,0x00,0x00,0x00,0x00,0x00,0x00},
    {0x36,0x36,0x7F,0x36,0x7F,0x36,0x36,0x00},
    {0x0C,0x3E,0x03,0x1E,0x30,0x1F,0x0C,0x00},
    {0x00,0x63,0x33,0x18,0x0C,0x66,0x63,0x00},
    {0x1C,0x36,0x1C,0x6E,0x3B,0x33,0x6E,0x00},
    {0x06,0x06,0x03,0x00,0x00,0x00,0x00,0x00},
    {0x18,0x0C,0x06,0x06,0x06,0x0C,0x18,0x00},
    {0x06,0x0C,0x18,0x18,0x18,0x0C,0x06,0x00},
    {0x00,0x66,0x3C,0xFF,0x3C,0x66,0x00,0x00},
    {0x00,0x0C,0x0C,0x3F,0x0C,0x0C,0x00,0x00},
    {0x00,0x00,0x00,0x00,0x00,0x0C,0x0C,0x06},
    {0x00,0x00,0x00,0x3F,0x00,0x00,0x00,0x00},
    {0x00,0x00,0x00,0x00,0x00,0x0C,0x0C,0x00},
    {0x60,0x30,0x18,0x0C,0x06,0x03,0x01,0x00},
    {0x3E,0x63,0x73,0x7B,0x6F,0x67,0x3E,0x00},
    {0x0C,0x0E,0x0C,0x0C,0x0C,0x0C,0x3F,0x00},
    {0x1E,0x33,0x30,0x1C,0x06,0x33,0x3F,0x00},
    {0x1E,0x33,0x30,0x1C,0x30,0x33,0x1E,0x00},
    {0x38,0x3C,0x36,0x33,0x7F,0x30,0x78,0x00},
    {0x3F,0x03,0x1F,0x30,0x30,0x33,0x1E,0x00},
    {0x1C,0x06,0x03,0x1F,0x33,0x33,0x1E,0x00},
    {0x3F,0x33,0x30,0x18,0x0C,0x0C,0x0C,0x00},
    {0x1E,0x33,0x33,0x1E,0x33,0x33,0x1E,0x00},
    {0x1E,0x33,0x33,0x3E,0x30,0x18,0x0E,0x00},
    {0x00,0x0C,0x0C,0x00,0x00,0x0C,0x0C,0x00},
    {0x00,0x0C,0x0C,0x00,0x00,0x0C,0x0C,0x06},
    {0x18,0x0C,0x06,0x03,0x06,0x0C,0x18,0x00},

```

```

    {0x00,0x00,0x3F,0x00,0x00,0x3F,0x00,0x00},
    {0x06,0x0C,0x18,0x30,0x18,0x0C,0x06,0x00},
    {0x1E,0x33,0x30,0x18,0x0C,0x00,0x0C,0x00},
    {0x3E,0x63,0x7B,0x7B,0x7B,0x03,0x1E,0x00},
    {0x0C,0x1E,0x33,0x33,0x3F,0x33,0x33,0x00},
    {0x3F,0x66,0x66,0x3E,0x66,0x66,0x3F,0x00},
    {0x3C,0x66,0x03,0x03,0x03,0x66,0x3C,0x00},
    {0x1F,0x36,0x66,0x66,0x66,0x36,0x1F,0x00},
    {0x7F,0x46,0x16,0x1E,0x16,0x46,0x7F,0x00},
    {0x7F,0x46,0x16,0x1E,0x16,0x06,0x0F,0x00},
    {0x3C,0x66,0x03,0x03,0x73,0x66,0x7C,0x00},
    {0x33,0x33,0x33,0x3F,0x33,0x33,0x33,0x00},
    {0x1E,0x0C,0x0C,0x0C,0x0C,0x0C,0x1E,0x00},
    {0x78,0x30,0x30,0x30,0x33,0x33,0x1E,0x00},
    {0x67,0x66,0x36,0x1E,0x36,0x66,0x67,0x00},
    {0x0F,0x06,0x06,0x06,0x46,0x66,0x7F,0x00},
    {0x63,0x77,0x7F,0x7F,0x6B,0x63,0x63,0x00},
    {0x63,0x67,0x6F,0x7B,0x73,0x63,0x63,0x00},
    {0x1C,0x36,0x63,0x63,0x63,0x36,0x1C,0x00},
    {0x3F,0x66,0x66,0x3E,0x06,0x06,0x0F,0x00},
    {0x1E,0x33,0x33,0x33,0x3B,0x1E,0x38,0x00},
    {0x3F,0x66,0x66,0x3E,0x36,0x66,0x67,0x00},
    {0x1E,0x33,0x07,0x0E,0x38,0x33,0x1E,0x00},
    {0x3F,0x2D,0x0C,0x0C,0x0C,0x0C,0x1E,0x00},
    {0x33,0x33,0x33,0x33,0x33,0x33,0x3F,0x00},
    {0x33,0x33,0x33,0x33,0x33,0x1E,0x0C,0x00},
    {0x63,0x63,0x63,0x6B,0x7F,0x77,0x63,0x00},
    {0x63,0x63,0x36,0x1C,0x1C,0x36,0x63,0x00},
    {0x33,0x33,0x33,0x1E,0x0C,0x0C,0x1E,0x00},
    {0x7F,0x63,0x31,0x18,0x4C,0x66,0x7F,0x00},
};

static inline void lcd_cs_select()    { gpio_put(LCD_CS, 0); sleep_us(1); }
static inline void lcd_cs_deselect() { sleep_us(1); gpio_put(LCD_CS, 1); }
static inline void lcd_dc_command()  { gpio_put(LCD_DC, 0); }
static inline void lcd_dc_data()     { gpio_put(LCD_DC, 1); }

void lcd_write_command(uint8_t cmd) {
    lcd_dc_command();
    lcd_cs_select();
    spi_write_blocking(LCD_SPI, &cmd, 1);
    lcd_cs_deselect();
}

void lcd_write_data(uint8_t data) {
    lcd_dc_data();

```

```

    lcd_cs_select();
    spi_write_blocking(LCD_SPI, &data, 1);
    lcd_cs_deselect();
}

void lcd_reset() {
    gpio_put(LCD_RESET, 1); sleep_ms(5);
    gpio_put(LCD_RESET, 0); sleep_ms(20);
    gpio_put(LCD_RESET, 1); sleep_ms(150);
}

void lcd_init() {
    lcd_reset();
    lcd_write_command(ILI9341_SWRESET); sleep_ms(150);
    lcd_write_command(ILI9341_SLPOUT); sleep_ms(120);
    lcd_write_command(ILI9341_PIXFMT); lcd_write_data(0x55);
    lcd_write_command(ILI9341_MADCTL); lcd_write_data(0x48);
    lcd_write_command(ILI9341_DISPON); sleep_ms(100);
    printf("LCD initialized\n");
}

void lcd_set_address_window(uint16_t x0, uint16_t y0, uint16_t x1,
uint16_t y1) {
    lcd_write_command(ILI9341_CASET);
    lcd_write_data(x0 >> 8); lcd_write_data(x0 & 0xFF);
    lcd_write_data(x1 >> 8); lcd_write_data(x1 & 0xFF);
    lcd_write_command(ILI9341_PASET);
    lcd_write_data(y0 >> 8); lcd_write_data(y0 & 0xFF);
    lcd_write_data(y1 >> 8); lcd_write_data(y1 & 0xFF);
    lcd_write_command(ILI9341_RAMWR);
}

void lcd_fill_rect(uint16_t x, uint16_t y, uint16_t w, uint16_t h,
uint16_t color) {
    if (x >= LCD_WIDTH || y >= LCD_HEIGHT) return;
    if (x + w > LCD_WIDTH) w = LCD_WIDTH - x;
    if (y + h > LCD_HEIGHT) h = LCD_HEIGHT - y;

    uint8_t buf[2] = {color >> 8, color & 0xFF};
    lcd_set_address_window(x, y, x + w - 1, y + h - 1);
    lcd_dc_data();
    lcd_cs_select();
    for (uint32_t i = 0; i < (uint32_t)w * h; i++) {
        spi_write_blocking(LCD_SPI, buf, 2);
    }
    lcd_cs_deselect();
}

```

```

}

void lcd_fill_screen(uint16_t color) {
    lcd_fill_rect(0, 0, LCD_WIDTH, LCD_HEIGHT, color);
}

void lcd_draw_char(uint16_t x, uint16_t y, char c, uint16_t color,
uint16_t bg, uint8_t scale) {
    if (c < 32 || c > 90) c = 32;
    const uint8_t *glyph = font8x8[c - 32];
    lcd_set_address_window(x, y, x + 8*scale - 1, y + 8*scale - 1);

    uint8_t chi = color >> 8, clo = color & 0xFF;
    uint8_t bhi = bg >> 8, blo = bg & 0xFF;

    lcd_dc_data();
    lcd_cs_select();
    for (uint8_t row = 0; row < 8; row++) {
        for (uint8_t sy = 0; sy < scale; sy++) {
            for (uint8_t col = 0; col < 8; col++) {
                bool on = glyph[row] & (1 << col);
                uint8_t px[2] = {on ? chi : bhi, on ? clo : blo};
                for (uint8_t sx = 0; sx < scale; sx++) {
                    spi_write_blocking(LCD_SPI, px, 2);
                }
            }
        }
    }
    lcd_cs_deselect();
}

void lcd_draw_string(uint16_t x, uint16_t y, const char *str, uint16_t
color, uint16_t bg, uint8_t scale) {
    while (*str) {
        lcd_draw_char(x, y, *str, color, bg, scale);
        x += 8 * scale;
        str++;
    }
}

void lcd_draw_space_status(int i, bool sensor_occupied) {
    uint16_t y = 65 + i * 50;

    bool show_occupied;
    if (space_type[i] == SPACE_UNAVAILABLE) {

```

```

        lcd_fill_rect(130, y - 2, 100, 22, COLOR_ORANGE);
        lcd_draw_string(155, y + 2, "N/A", COLOR_BLACK, COLOR_ORANGE, 2);
        return;
    } else if (override_active[i]) {
        show_occupied = override_occupied[i];
    } else {
        show_occupied = sensor_occupied;
    }

    if (show_occupied) {
        lcd_fill_rect(130, y - 2, 100, 22, COLOR_RED);
        lcd_draw_string(143, y + 2, "FULL", COLOR_WHITE, COLOR_RED, 2);
    } else if (space_type[i] == SPACE_HANDICAP) {
        lcd_fill_rect(130, y - 2, 100, 22, COLOR_BLUE);
        lcd_draw_string(143, y + 2, "OPEN", COLOR_WHITE, COLOR_BLUE, 2);
    } else {
        lcd_fill_rect(130, y - 2, 100, 22, COLOR_GREEN);
        lcd_draw_string(143, y + 2, "OPEN", COLOR_BLACK, COLOR_GREEN, 2);
    }
}

void lcd_draw_parking_screen(bool occupied[NUM_SPACES]) {
    lcd_fill_screen(COLOR_BLACK);
    lcd_draw_string(20, 10, "SMART PARKING", COLOR_CYAN, COLOR_BLACK, 2);
    lcd_draw_string(30, 35, "PURDUE IUPUI", COLOR_GRAY, COLOR_BLACK, 1);
    lcd_fill_rect(0, 52, LCD_WIDTH, 2, COLOR_GRAY);

    for (int i = 0; i < NUM_SPACES; i++) {
        uint16_t y = 65 + i * 50;
        char label[12];

        if (space_type[i] == SPACE_HANDICAP) {
            snprintf(label, sizeof(label), "SP%d HC", i + 1);
            lcd_draw_string(10, y, label, COLOR_CYAN, COLOR_BLACK, 2);
        } else if (space_type[i] == SPACE_UNAVAILABLE) {
            snprintf(label, sizeof(label), "SP%d", i + 1);
            lcd_draw_string(10, y, label, COLOR_GRAY, COLOR_BLACK, 2);
        } else {
            snprintf(label, sizeof(label), "SP%d", i + 1);
            lcd_draw_string(10, y, label, COLOR_WHITE, COLOR_BLACK, 2);
        }
        lcd_draw_space_status(i, occupied[i]);
    }

    lcd_fill_rect(0, 315, LCD_WIDTH, 2, COLOR_GRAY);
}

```

```

    int free_count = 0, total_usable = 0;
    for (int i = 0; i < NUM_SPACES; i++) {
        if (space_type[i] != SPACE_UNAVAILABLE) {
            total_usable++;
            if (!occupied[i]) free_count++;
        }
    }
    char summary[24];
    snprintf(summary, sizeof(summary), "AVAILABLE: %d/%d", free_count,
total_usable);
    lcd_draw_string(30, 320, summary, COLOR_YELLOW, COLOR_BLACK, 1);
}

void lcd_update_changed(bool current_state[NUM_SPACES], bool
force_label[NUM_SPACES]) {
    for (int i = 0; i < NUM_SPACES; i++) {
        bool display_state = override_active[i] ? override_occupied[i] :
current_state[i];
        bool display_changed = (display_state != last_state[i]) ||
force_label[i];

        if (!display_changed) continue;

        last_state[i] = display_state;

        if (force_label[i]) {
            uint16_t y = 65 + i * 50;
            lcd_fill_rect(0, y - 2, 128, 22, COLOR_BLACK);
            char label[12];
            if (space_type[i] == SPACE_HANDICAP) {
                snprintf(label, sizeof(label), "SP%d HC", i + 1);
                lcd_draw_string(10, y, label, COLOR_CYAN, COLOR_BLACK, 2);
            } else if (space_type[i] == SPACE_UNAVAILABLE) {
                snprintf(label, sizeof(label), "SP%d", i + 1);
                lcd_draw_string(10, y, label, COLOR_GRAY, COLOR_BLACK, 2);
            } else {
                snprintf(label, sizeof(label), "SP%d", i + 1);
                lcd_draw_string(10, y, label, COLOR_WHITE, COLOR_BLACK,
2);
            }
        }
        lcd_draw_space_status(i, current_state[i]);
    }
}

```

```

    int free_count = 0, total_usable = 0;
    for (int i = 0; i < NUM_SPACES; i++) {
        if (space_type[i] != SPACE_UNAVAILABLE) {
            total_usable++;
            bool disp = override_active[i] ? override_occupied[i] :
current_state[i];
            if (!disp) free_count++;
        }
    }
    lcd_fill_rect(0, 318, LCD_WIDTH, 16, COLOR_BLACK);
    char summary[24];
    snprintf(summary, sizeof(summary), "AVAILABLE: %d/%d", free_count,
total_usable);
    lcd_draw_string(30, 320, summary, COLOR_YELLOW, COLOR_BLACK, 1);
}

void led_set_color(int i, bool r, bool g, bool b) {
    gpio_put(led_pins[i][0], r);
    gpio_put(led_pins[i][1], g);
    gpio_put(led_pins[i][2], b);
}

void led_set_free(int i)           { led_set_color(i, 0, 1, 0); }
void led_set_handicap_free(int i) { led_set_color(i, 0, 0, 1); }
void led_set_occupied(int i)       { led_set_color(i, 1, 0, 0); }
void led_set_maintenance(int i)    { led_set_color(i, 1, 1, 0); }

void led_apply_state(int i, bool sensor_occupied) {
    if (space_type[i] == SPACE_UNAVAILABLE) {
        led_set_maintenance(i);
        return;
    }
    bool show_occupied = override_active[i] ? override_occupied[i] :
sensor_occupied;
    if (show_occupied) {
        led_set_occupied(i);
    } else if (space_type[i] == SPACE_HANDICAP) {
        led_set_handicap_free(i);
    } else {
        led_set_free(i);
    }
}

bool space_is_occupied(int i) {
    if (space_type[i] == SPACE_UNAVAILABLE) return false;

```



```

        return !gpio_get(sensor_pins[i]);
    }

void sensors_init() {
    for (int i = 0; i < NUM_SPACES; i++) {
        gpio_init(sensor_pins[i]);
        gpio_set_dir(sensor_pins[i], GPIO_IN);
    }
}

void buttons_init() {
    for (int i = 0; i < NUM_SPACES; i++) {
        gpio_init(button_pins[i]);
        gpio_set_dir(button_pins[i], GPIO_IN);
        gpio_pull_up(button_pins[i]);
    }
}

void leds_init() {
    for (int i = 0; i < NUM_SPACES; i++) {
        for (int c = 0; c < 3; c++) {
            gpio_init(led_pins[i][c]);
            gpio_set_dir(led_pins[i][c], GPIO_OUT);
            gpio_put(led_pins[i][c], 0);
        }
    }
}

void lcd_spi_init() {
    spi_init(LCD_SPI, 62500000);
    spi_set_format(LCD_SPI, 8, SPI_CPOL_0, SPI_CPHA_0, SPI_MSB_FIRST);
    gpio_set_function(LCD_MOSI, GPIO_FUNC_SPI);
    gpio_set_function(LCD_SCK, GPIO_FUNC_SPI);
    gpio_init(LCD_CS);    gpio_set_dir(LCD_CS, GPIO_OUT); gpio_put(LCD_CS,
1);
    gpio_init(LCD_DC);    gpio_set_dir(LCD_DC, GPIO_OUT); gpio_put(LCD_DC,
1);
    gpio_init(LCD_RESET); gpio_set_dir(LCD_RESET, GPIO_OUT);
    gpio_put(LCD_RESET, 1);
    gpio_init(LCD_LED);   gpio_set_dir(LCD_LED, GPIO_OUT);
    gpio_put(LCD_LED, 1);
}

bool process_button_for_space(int btn, int space, bool sensor_occupied,
bool *force_label) {
    bool pressed = !gpio_get(button_pins[btn]);

```

```

bool changed = false;
*force_label = false;

if (pressed && !btn_was_pressed[btn]) {
    btn_press_time[btn] = to_ms_since_boot(get_absolute_time());
    btn_was_pressed[btn] = true;
    btn_hold_fired[btn] = false;
}

if (pressed && btn_was_pressed[btn] && !btn_hold_fired[btn]) {
    uint32_t held_ms = to_ms_since_boot(get_absolute_time()) -
btn_press_time[btn];
    if (held_ms >= HOLD_THRESHOLD_MS) {
        btn_hold_fired[btn] = true;
        if (space_type[space] == SPACE_UNAVAILABLE) {
            space_type[space] = saved_type[space];
            override_active[space] = false;
        } else {
            saved_type[space] = space_type[space];
            space_type[space] = SPACE_UNAVAILABLE;
            override_active[space] = false;
        }
        *force_label = true;
        changed = true;
    }
}

if (!pressed && btn_was_pressed[btn]) {
    uint32_t held_ms = to_ms_since_boot(get_absolute_time()) -
btn_press_time[btn];
    btn_was_pressed[btn] = false;

    if (!btn_hold_fired[btn] && held_ms >= DEBOUNCE_MS) {
        if (space_type[space] == SPACE_UNAVAILABLE) {
            space_type[space] = saved_type[space];
            override_active[space] = false;
            *force_label = true;
        } else if (!override_active[space]) {
            override_active[space] = true;
            override_occupied[space] = !sensor_occupied;
        } else {
            override_active[space] = false;
        }
        changed = true;
    }
}

```

```

        return changed;
    }

void print_parking_stats(bool current_occupied[NUM_SPACES]) {
    uint32_t now = to_ms_since_boot(get_absolute_time());
    if (now - last_print_time < 5000) return;
    last_print_time = now;

    printf("\n=== Parking System Status ===\n");
    int free_count = 0;
    int recommended = -1;
    uint32_t best_avg_sec = 0;

    for (int i = 0; i < NUM_SPACES; i++) {
        bool is_occupied = current_occupied[i];

        uint32_t display_total_ms = total_occupied_ms[i];
        uint32_t display_count = occupation_count[i];
        if (is_occupied) {
            uint32_t live_ms = (uint32_t)(absolute_time_diff_us(
                occupation_start_time[i],
                get_absolute_time()) / 1000);

            if (live_ms > 0) {
                display_total_ms += live_ms;
                display_count++;
            }
        }

        uint32_t avg_sec = (display_count > 0)
            ? (display_total_ms / display_count) / 1000
            : 0;

        if (space_type[i] == SPACE_UNAVAILABLE) {
            printf("Space %d (N/A) : UNAVAILABLE\n", i + 1);
        } else {
            printf("Space %d (%-6s): %-8s | Avg occupied: %3lu sec (%lu
session%s)\n",
                i + 1,
                (space_type[i] == SPACE_HANDICAP) ? "HC" : "Normal",
                is_occupied ? "OCCUPIED" : "FREE",
                avg_sec,
                display_count,
                display_count == 1 ? "" : "s");
        }
    }
}

```

```

        if (!is_occupied) free_count++;

        if (!is_occupied && space_type[i] == SPACE_NORMAL) {
            if (display_count > 0 && avg_sec > best_avg_sec) {
                best_avg_sec = avg_sec;
                recommended = i;
            }
        }
    }

    if (recommended == -1) {
        for (int i = 0; i < NUM_SPACES; i++) {
            if (!current_occupied[i] && space_type[i] == SPACE_NORMAL) {
                recommended = i;
                break;
            }
        }
    }

    printf("-----\n");
    if (recommended != -1) {
        uint32_t rec_count = occupation_count[recommended];
        uint32_t rec_avg = (rec_count > 0)
            ? (total_occupied_ms[recommended] /
rec_count) / 1000
            : 0;
        if (rec_count > 0) {
            printf(">> RECOMMENDED: Space %d (avg %lu sec/visit - lowest
turnover, least congestion)\n",
                recommended + 1, rec_avg);
        } else {
            printf(">> RECOMMENDED: Space %d (no history yet - first
available normal space)\n",
                recommended + 1);
        }
    } else {
        printf(">> No free standard spaces available.\n");
    }

    printf("Free spaces: %d/%d\n", free_count, NUM_SPACES);
    printf("=====\n\n");
}

```

```

int main() {
    stdio_init_all();
    sleep_ms(3000);

    printf("=== Smart Parking System Started ===\n");

    sensors_init();
    buttons_init();
    leds_init();
    lcd_spi_init();
    lcd_init();

    bool current_state[NUM_SPACES];
    for (int i = 0; i < NUM_SPACES; i++) {
        current_state[i] = space_is_occupied(i);
        last_state[i] = current_state[i];
        occupation_start_time[i] = get_absolute_time();
    }

    for (int i = 0; i < NUM_SPACES; i++) {
        led_apply_state(i, current_state[i]);
    }

    lcd_draw_parking_screen(current_state);

    printf("System ready. Console will show stats every 5 seconds.\n\n");

    while (true) {
        bool changed = false;
        bool force_label[NUM_SPACES] = {false};

        bool sensor_occupied[NUM_SPACES];
        for (int i = 0; i < NUM_SPACES; i++) {
            sensor_occupied[i] = space_is_occupied(i);
        }

        for (int btn = 0; btn < NUM_SPACES; btn++) {
            int space = button_controls_space[btn];
            bool lbl = false;
            if (process_button_for_space(btn, space,
sensor_occupied[space], &lbl)) {
                changed = true;
            }
        }
    }
}

```

```

        force_label[space] = lbl;
    }
}

for (int i = 0; i < NUM_SPACES; i++) {
    if (sensor_occupied[i] && !current_state[i]) {

        occupation_start_time[i] = get_absolute_time();
    } else if (!sensor_occupied[i] && current_state[i]) {

        uint32_t duration_ms =
absolute_time_diff_us(occupation_start_time[i], get_absolute_time()) /
1000;

        if (duration_ms > 1000) {
            total_occupied_ms[i] += duration_ms;
            occupation_count[i]++;
        }
    }

    led_apply_state(i, sensor_occupied[i]);

    if (sensor_occupied[i] != current_state[i]) {
        current_state[i] = sensor_occupied[i];
        changed = true;
    }
}

if (changed) {
    lcd_update_changed(current_state, force_label);
}

print_parking_stats(current_state);

sleep_ms(50);
}

return 0;
}

```